

MPhys Lab Book

Benjamin M.

December 9, 2025

Contents

1	Week 1	3
1.1	Day 1 : Tuesday, 30 September	3
1.1.1	Paper 1 Notes	3
1.1.2	General Comments	4
1.2	Day 2 : Thursday, 2 October	5
1.2.1	CGCNN Structure	5
2	Week 2	8
2.1	Day 3 : Tuesday, 7 October	8
2.2	Day 4 : Thursday, 9 October	9
3	Week 3 [INCOMPLETE!]	11
3.1	Day 5 : Tuesday, 14 October	11
3.2	Day 6 : Thursday, 16 October	12
4	Week 4 (INCOMPLETE!!!)	13
4.1	Saturday, 18 October	13
4.2	Day 7 : Tuesday, 21 October	15
4.3	Day 8: Thursday, 23 October	16
5	Week 5 (INCOMPLETE!!!)	20
5.1	Day 9 : Tuesday, 28 October	20
5.2	Day 10 : Thursday, 30 October	21
6	Week 6 (INCOMPLETE!!)	24
6.1	Monday, 3 November	24
6.2	Day 11 : Tuesday, 4 November	25
6.3	Day 12 : Thursday, 6 November	30
7	Week 7 (check it!!)	31
7.1	Day 13 : Tuesday, 11 November	31
7.2	Day 14 : Thursday, 13 November	33
8	Week 8 (INCOMPLETE!)	35
8.1	Monday, 17 November	35
8.2	Day 15 : Tuesday, 18 November	35
8.3	Day 16 : Thursday, 19 November	36
9	Week 9	37
9.1	Day 17 : Tuesday, 25 November	37
9.2	Wednesday, 26 November	39
9.3	Day 18 : Thursday, 27 November	41
9.4	Friday, 28 November	43
9.5	Saturday, 28 November	43
9.6	Monday, 1 December	44
10	Week 10 (missing anything after Day 1)	45
10.1	Day 19 : Tuesday, 2 December	45

1 Week 1

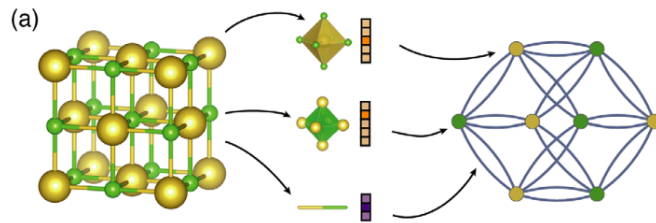
1.1 Day 1 : Tuesday, 30 September

Met with supervisor, assigned a number of papers to read through, initially:

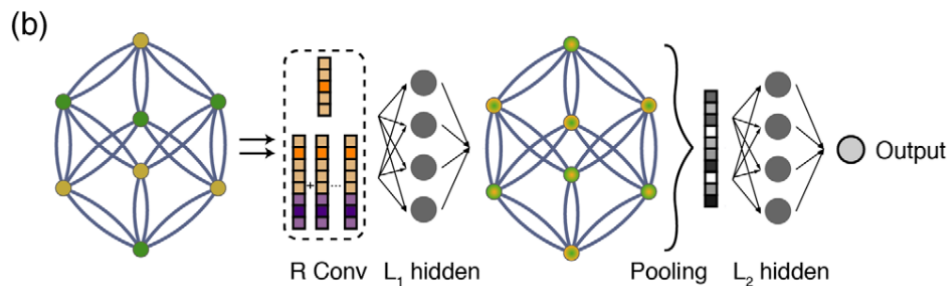
1. <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.120.145301> (Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties)
2. <https://www.nature.com/articles/s41524-021-00650-1> (Atomistic Line Graph Neural Network for improved materials property predictions)

1.1.1 Paper 1 Notes

- Most ML methods struggle with modelling infinite crystalline structure...
 - ⇒ Represent the structure with a crystal graph, which encodes both atomic info, and bonding interactions between atoms.
 - ⇒ Nodes represent atoms, edges represent bonds between atoms; we **allow** for multiple edges between nodes to account for periodicity.



- ⇒ Each node is represented by a feature vector, $\mathbf{v}_i$, which encodes properties of the atom at that node. Each edge is represented by another feature vector $\mathbf{u}_{(i,j)_k}$ (k th bond between atoms i and j).
- Build convolutional NNs on top of these crystal graphs - achieves good level of accuracy to DFT (density functional theory) calculations.
 - ⇒ Composed of two main components: convolution layers and pooling layers.



- ⇒ Convolution layer iteratively updates $\mathbf{v}_i$ with some **nonlinear** convolution function of surrounding atoms and bonds,

$$\mathbf{v}_i^{t+1} = \text{Conv}(\mathbf{v}_i^{(t)}, \mathbf{v}_j^{(t)}, \mathbf{u}_{(i,j)_k})$$

- ⇒ The pooling layer then produces some overall crystal feature vector, $\mathbf{v}_c$, via some pooling function of the convolutions,

$$\mathbf{v}_c = \text{Pool}(\mathbf{v}_0^{(0)}, \mathbf{v}_1^{(0)}, \dots, \mathbf{v}_N^{(0)}, \dots, \mathbf{v}_N^{(R)}),$$

which should satisfy permutational invariance wrt atom indexing, and size invariance wrt unit cell choice. They use a normalised summation as the pooling function.

$$\text{Softmax}(z)_j = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

$$\tilde{\mathbf{v}}_i^{(t)} = \text{Softmax}(\mathbf{W}_t \mathbf{v}_i^{(t)} + \mathbf{b}_t)$$

$$\mathbf{v}_c = \sum_{t,i} \tilde{\mathbf{v}}_i^{(t)}$$

- ⇒ In addition, there are two *fully connected hidden layers*, with depths L_1 and L_2 - what are these? What do they achieve? Supposedly it helps with capturing complex mapping.
- Training is performed by minimising the difference between predicted properties and DFT calculated properties.
 - ⇒ The whole network can be considered as some function, f , parameterised by a set of weights, $\mathbf{W}$, which map a given crystal to some target property.
 - ⇒ Thus, we try to minimise the cost function wrt these weights, with the hope that the learned weights can be used to predict material properties. We use SGD to optimise for the weights.
 - ⇒ They train the NN on the full materials database (excluding 'ill-converged' crystals).
- An effective convolution function is found in terms of the concatenation of feature vectors of neighbours of i , $\mathbf{z}_{(i,j)_k}^{(t)} = \mathbf{v}_i^{(t)} \oplus \mathbf{v}_j^{(t)} \oplus \mathbf{u}_{(i,j)_k}$, with

$$\mathbf{v}_i^{(t+1)} = \mathbf{v}_i^{(t)} + \sum_{j,k} \sigma\left(\mathbf{z}_{(i,j)_k}^{(t)} \mathbf{W}_f^{(t)} + \mathbf{b}_f^{(t)}\right) \odot g\left(\mathbf{z}_{(i,j)_k} \mathbf{W}_s^{(t)} + \mathbf{b}_s^{(t)}\right),$$

where $\oplus$ is element-wise multiplication, and σ is a sigmoid (softmax) function.

- Best MAEs achieved with this are 0.039 eV/atom.
- Does well on predicting band gaps.

1.1.2 General Comments

I won't spend too much time talking about the other papers, since the above is the most relevant for the time being. For completeness, the other papers I read through were:

- <https://advanced.onlinelibrary.wiley.com/doi/10.1002/advs.201801367>
 - ⇒ A review of several deep learning models (MLP, CNN, and DTNN) applied for finding excitation spectra of molecules.
 - ⇒ The NNs find the eigenvalues, and spectra are estimated with Gaussian broadening (of these eigenvalues).
- <https://pubs.acs.org/doi/10.1021/acs.jpcclett.5c00169>
 - ⇒ Applying transfer learning methodologies to predict Raman spectra for complicated peptides and proteins.
- <https://www.mdpi.com/2673-4532/3/3/20>
 - ⇒ A nice review paper of various deep learning methods. I used this as a foundation for my understanding of the CNN (convolutional neural network).
- <https://www.nature.com/articles/s41597-025-04593-w>

⇒ The dataset we will be using for the purposes of this project. Once we make the 2D model, we'll use this as training data.

1.2 Day 2 : Thursday, 2 October

After meeting with our supervisor, we decided to proceed for now with the goal:

Understand the CGCNN model, and attempt to apply it to 2D materials.

The first step, then, was to find the program that the paper uses; conveniently, this was all stored in the following GitHub repo:

CGCNN repo: <https://github.com/txie-93/cgcnn>

I forked this and stored it in our personal repo:

Personal repo: https://github.com/bd-mat/raman_spectra

Before running this program, we found that it was necessary to install the pytorch and pymatgen packages. I had some issues with this step due to the large number of unused python installations on my computer - in the end, I chose to use VSCode as the IDE for this project, and I'm reusing the Spyder kernel to make things easier.

To run the CGCNN, we used their pre-trained models (CGCNN/pre-trained) with `predict.py`, which has a feature to load them in and make predictions based on the quantity they are trained on.

- We initially had some issues working with this program as a jupyter notebook - it relies on inputs put in via the command line when the file is called.
- ⇒ As such, we made some changes to the program, and introduced a 'utility class', which simply stores all of our initial information, and allows it to be called in a similar way to the original program parsers:

```
class inits:
    def __init__(self):
        self.modelpath = 'C:/Users/bjama/Desktop/MPhys/CGCNN/pre-trained/formation-energy-per-atom.pth.tar'
        self.cifpath = 'C:/Users/bjama/Desktop/MPhys/CGCNN/data/sample-regression'
        self.batch_size = 256
        self.workers = 0
        self.disable_cuda = True
        self.print_freq = 10
```

A messy solution, but it works!

- Initial runs seemed to be returning extremely large MAE values (~ 3 eV for the band gap) - we were unsure why and thought it might be a flaw with the loading of weights.

1.2.1 CGCNN Structure

Atomic data is stored in .cif files, and loaded into the model with `cgcnn\data.py`. The CGCNN model is contained in `cgcnn\model.py`, and has the following basic structure:

1. One embedding layer.

⇒ This is a linear layer, so transforms inputs x to output y as some

$$y = xA^T + b,$$

where A^T is a matrix of (trainable) weights, and b is a vector of (trainable) biases.

- ⇒ Importantly, this layer acts as a *bottleneck*, and serves to transform $x \rightarrow y$, where the dimensionality of y is **lower** than x . Here, we take an input feature vector of length 92, and return an output vector of length 64, for each atom.
- ⇒ The input, x , is a sparse vector of integer values (binary, in fact), while the output is continuously valued (and dense).

2. ~ 4 **convolution layers**.

- ⇒ Each layer applies the convolution function from the CGCNN paper,

$$\mathbf{v}_i^{(t+1)} = \mathbf{v}_i^{(t)} + \sum_{j,k} \sigma\left(\mathbf{z}_{(i,j)_k}^{(t)} \mathbf{W}_f^{(t)} + \mathbf{b}_f^{(t)}\right) \odot g\left(\mathbf{z}_{(i,j)_k}^{(t)} \mathbf{W}_s^{(t)} + \mathbf{b}_s^{(t)}\right), \quad (1)$$

given inputs for each atom of $(\text{atom_feature_vector}, \text{nbr_bond_vectors}, \text{nbr_indices})$. The atom feature vector in question has *already* passed through the embedding layer, so is just that for the first layer, and in further convolution layers, is simply the output of the previous layer.

- ⇒ Obviously, the atom feature vector is the only input to the convolution function which actually changes through these layers, so the other two inputs are ‘effectively’ constants.
- ⇒ We don’t really need any more than 4 convolution layers, since our graph is relatively small, so this is sufficient for an atom to ‘learn’ about all other atoms in the structure.

3. **One pooling layer**.

- ⇒ We pool the set of *atom* feature vectors to a single *crystal* feature vector, which is achieved by just summing them [?]. The output has the same length as a single atom feature vector.

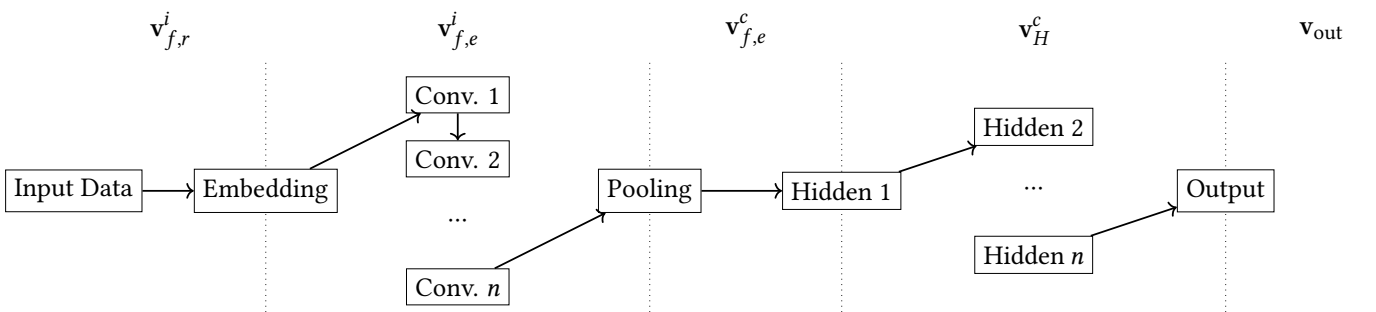
4. **Hidden layers**.

- ⇒ The output of the pooling layer is then passed through a series of hidden layers, each of which are modelled as linear transformations, which are then passed through a *softplus* function, before being sent to the next hidden layer.
- ⇒ The hidden feature vectors have a **different** length to the crystal feature vector, and so the first layer (which is always there) acts in a similar way to the embedding layer, changing the size of the feature vector from 64 to some number, e.g. 32, which is then unchanged through the rest of the hidden layers.

5. **Output layer**.

- ⇒ Finally, the final hidden feature vector (of length 32) is passed to the output layer, which is again **linear**, and now maps to the **final** output of the neural network.
- ⇒ If we are estimating a single quantity, the output is just a scalar. Potentially, we can model multiple outputs by just increasing this value? Will require some testing 😊.

I will summarise the CGCNN structure in the following diagram:



Hopefully, I can now paste the definitions of the main CGCNN class, and the action of each line will be obvious:

Firstly, the initialisation of the class:

```
super(CrystalGraphConvNet, self).__init__()
self.embedding = nn.Linear(orig_atom_fea_len, atom_fea_len)
self.convs = nn.ModuleList([ConvLayer(atom_fea_len=atom_fea_len,
                                      nbr_fea_len=nbr_fea_len)
                           for _ in range(n_conv)])
self.conv_to_fc = nn.Linear(atom_fea_len, h_fea_len)
self.conv_to_fc_softplus = nn.Softplus()
if n_h > 1:
    self.fcs = nn.ModuleList([nn.Linear(h_fea_len, h_fea_len)
                              for _ in range(n_h-1)])
    self.softpluses = nn.ModuleList([nn.Softplus()
                                      for _ in range(n_h-1)])
self.fc_out = nn.Linear(h_fea_len, 1)
```

Secondly, the forward function (i.e. action of the NN):

```
atom_fea = self.embedding(atom_fea)
for conv_func in self.convs:
    atom_fea = conv_func(atom_fea, nbr_fea, nbr_fea_idx)
crys_fea = self.pooling(atom_fea, crystal_atom_idx)
crys_fea = self.conv_to_fc(self.conv_to_fc_softplus(crys_fea))
crys_fea = self.conv_to_fc_softplus(crys_fea)
if hasattr(self, 'fcs') and hasattr(self, 'softpluses'):
    for fc, softplus in zip(self.fcs, self.softpluses):
        crys_fea = softplus(fc(crys_fea))
out = self.fc_out(crys_fea)
return out
```

I will go into more detail into the pooling and convolution parts next week (hopefully).

2 Week 2

2.1 Day 3 : Tuesday, 7 October

We successfully identified the embedding layer of the CGCNN, which is a linear layer, taking in the original atom feature length (92), and returning the model feature length (64):

```
[65] encoder = nn.Linear(orig_atom_fea_len,model_args.atom_fea_len)
```

This is taken from `predict.py`. Furthermore, we identified the step which loads (pre-trained) model weights as:

```
model.load_state_dict(checkpoint['state_dict'])
```

Next, we downloaded the 2dmatpedia dataset (<https://www.2dmatpedia.org/>), and attempted to transfer it to a workable format:

- The initial data is a single, large (25Mb) json file, where each line contains the information for a single crystal.
- The CGCNN program loads `.cif` (crystal information file) files (via `data.py`), which contain structural information on the crystals.
 - ⇒ After reading through their data loading program, we noticed that all of the info ported from a cif file was stored in a pymatgen *Structure* object.
- As it turned out, there was a portion of each crystal JSON which corresponded exactly to the information in these objects!
 - ⇒ I wrote a program (`data.py`, NOT in CGCNN folder), which loaded the entire JSON file with pandas (call this data 'test'), and isolated 'test.structure', outputting the structural data of each crystal to a **separate** file.
 - ⇒ These files were stored in the 'structure_database' folder, and their names were simply set to the material IDs, for easy indexing.

```
import pandas as pd
import numpy as np

test = pd.read_json('2dmatpedia/db.json',lines=True)
structs = test.structure
ids = test.material_id
# writing structure files

for i in range(0,len(structs)):
    file = open('structure_database/'+ids[i]+'.json','w')
    file.write(str(structs[i]).replace("'",''))
    file.close()
```

A simple program to extract the structure information and store it. Note that the step of replacing single quotes with double quotes is necessary to conform to JSON formatting!

- The pymatgen *Structure* classes could then be loaded **directly** from these files with the 'Structure.from_file()' utility.

Finally, we had a brief look at how the program encoded the initial feature vectors - the results were not what we expected. Within the directory of cif files, there is an 'atom_init.json', which maps a given atomic number (up to 100) to a 92-length vector of *binary* values - the program simply matches each atom to a vector using this file.

As for the elements of this vector, the CGCNN paper states that they are allocated as:

Property	Unit	Range	# of categories
Group number	–	1,2, ..., 18	18
Period number	–	1,2, ..., 9 ^a	9
Electronegativity[7, 8]	–	0.5–4.0	10
Covalent radius[3]	pm	25–250	10
Valence electrons	–	1, 2, ..., 12	12
First ionization energy[9] ^b	eV	1.3–3.3	10
Electron affinity[10]	eV	–3–3.7	10
Block	–	s, p, d, f	4
Atomic volume ^b	cm ³ /mol	1.5–4.3	10

^a The lanthanide and actinide elements are considered as period 8 and 9 respectively.

^b Log scale is used for these properties.

⇒ After speaking with our supervisor, she stated that the reason for this 'one-hot' allocation is to ensure that the model does not assume some false linear relationship between atomic numbers, or other properties of the atom.

2.2 Day 4 : Thursday, 9 October

Having processed the set of crystal structures, we wanted to attempt to run the CGCNN on our 2D dataset. We figured that this could be achieved in a few steps:

1. Identify a property of the crystals to predict.
2. Extract these properties to a file.
3. Modify the CGCNN data.py to load JSON files, and work with our custom dataset.
4. Observe results.

For step 1:

- While it would be nice to immediately predict energy eigenvalues - we found that this was **not** included in the data we had downloaded (more on this later).

⇒ As such, we decided to go with the band gaps, for which there was already an existing model.

For step 2:

- We noticed that the target properties were stored in the same directory as the cif files, in a file called 'id_prop.csv'. True to its name, this is a file containing, on one column, the material IDs, and on the other, the values of the target property (i.e. CGCNN output) for each one.

⇒ This was relatively simple to implement, and I wrote it into the same file used for producing the structure dataset:

```
# writing id_prop.csv
file = open('structure_database/id_prop.csv','w')
for i in range(0,len(bandgaps)):
    file.write(str(ids[i])+' '+str(bandgaps[i])+"\n")
file.close()
```

Step 3, which we thought might be simple, turned out to pose the largest problems:

- We initially wanted to run the CGCNN for a small subset of the 2dmatpedia dataset - we searched data.py for any mentions of the cif file format, and found that the only one was in the definition of the CIFdata() function (unsurprisingly):

```
[333] crystal = Structure.from_file(os.path.join(self.root_dir, cif_id+'.cif'))
```

- Our first thought was then, since the Structure variable would be identical once loaded, all we needed to do was changed the file ending to '.json'.

⇒ Surprisingly, this **did not work**. We verified that the dataset was loaded correctly, but the DataLoader object defined in predict.py caused some problems:

```
[56] test_loader = DataLoader(dataset, batch_size=args.batch_size, shuffle=True,
    num_workers=args.workers, collate_fn=collate_fn,
    pin_memory=args.cuda)
```

⇒ This **always** called an error when used in the validate() function, where we iterate over its properties as:

```
[142] for i, (input, target, batch_cif_ids) in enumerate(test_loader):
    ...
```

⇒ Generally, these errors would point to files that didn't exist, and it seemed as if there was some strange behaviour simply baked into the model.

We decided that it would be more efficient, at this point, to produce our own JSONdata class, which we would work on moving into the coming week.

3 Week 3 [INCOMPLETE!]

3.1 Day 5 : Tuesday, 14 October

Over the weekend, we finished the JSONData loader (see `json_data_loader.py`), which turned out to be relatively simple. Some troubleshooting revealed that the `.from_file` method worked fine for loading JSON files, so almost all of the code could simply be copied from the CGCNN, and the only line(s) that required changing were in the creation of the Structure object.

We ran a pre-trained band-gap model on a subset of the 2dmatpedia database, and it *did* run successfully, however, the MAE was around 0.6 eV, which is relatively large.

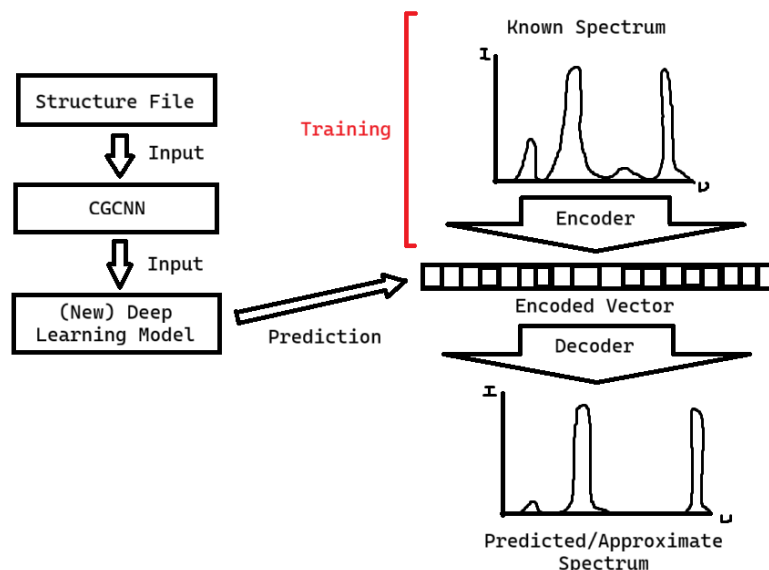
⇒ We shouldn't expect the pre-trained model to work too well on our dataset, since it has not been trained on purely 2D materials.

In that sense, we would like to train a new model to predict, e.g. the band gaps, of our 2D dataset.

We met with our supervisor to discuss next steps for the project:

- Ultimately, our goal is to predict Raman spectra given an input (2D) crystal structure.

⇒ This can be achieved with the CGCNN as a starting point by seeking to create something with the following architecture:



⇒ That is, the output of the CGCNN is fed into *another* neural network, which then predicts an encoded representation of the known Raman spectrum.

- We believe that the encoder could be trained on the DFT-predicted Raman spectra by:
 1. Inputting the entire known spectrum.
 2. Choosing an encoded length **much** less than the bins on the spectrum.
 3. Choosing the output to be the input spectrum, and a loss function which prioritises the feature we are most interested in (e.g. prominent peaks).
- Once the encoder is trained, we can convert our entire dataset to a set of encoded vectors, and use these to train the main deep learning model, which will be the entirety of our process.

One issue we had noticed was that our dataset (apparently) did not contain any Raman spectrum data - our supervisor clarified that we had not yet downloaded the actual Raman spectrum data. In fact, we had simply opened the first link in the paper and assumed it was the correct dataset (it was not). The relevant links were actually:

- <https://archive.materialscloud.org/record/2024.148>
- <http://mathtc.nsc-tj.cn/raman/>

There are a few crucial differences between this dataset and 2Dmatpedia:

- There are fewer crystals - 3504 down from 6531, but these are a subset of the 2Dmatpedia data.
- The structures are stored in a different format, POSCARs. These can also be loaded directly into pymatgen, but they require to be stored in a separate subfolder for each crystal since they must **always** be named 'POSCAR'.
- The Raman eigenvalues and intensities are stored in a separate file, `raman.dat`, for each crystal.

There is no need to change any of our code so far, since we can always write a pretty trivial POSCAR to JSON converter; pymatgen has a handy function which allows for you to directly save in the JSON format, `.to()`, and any other format, in fact.

The first step for this would be to convert our current crystal structures to 'single' 2D layers. We noted that, in the dataset we had downloaded, while there were indeed 2D layers, they were still periodic in the normal axis, with a comparable separation. To remedy this, we make a few observations about the dataset:

- The third lattice vector always corresponds to separation between layers.
- The third component of this lattice vector is generally the inter-layer separation.

It would follow then, that if we artificially increased the third lattice vector, v_3 , to reflect some very large separation (e.g. 20 times larger), then this would be a sufficient representation of a single 2D layer.

We implemented a short program to do this - the file was read in, and the file string was directly altered to multiply each element of the third lattice vector by a scaling factor, and finally, this was saved to an 'adjusted' structure JSON.

3.2 Day 6 : Thursday, 16 October

Given our dataset of adjusted structure files, our goal for the day was to attempt to gain experience in training models.

...

4 Week 4 (INCOMPLETE!!!)

4.1 Saturday, 18 October

We have a structure database for our crystals, however, they are currently 3D crystals: that is, each unit cell encodes a single layer, but the ‘vertical’ separation between layers is small enough that neighbours from the upper layers will still be considered in the structure embedding (since they lie within the valid range).

One key consideration allows this method to be automated:

⇒ The third lattice vector **always** corresponds to the periodicity between layers.

I came up with the following process for performing this:

1. Scale the third lattice vector by a factor 20 (this is sufficiently large).
2. The POSCAR files define the atomic positions in terms of lattice vector coefficients...

⇒ To retain the relative separation of atoms, we *also* scale their third lattice vector coefficient by a factor 1/20.

I implemented this in the program `datatransfer.py`. Note that, since there is no convenient method for importing POSCAR data, I manually input the files. Given the contents of such a file, content, it goes as:

```
sep = '   '
lines = content.split('\n')
# c vector:
c = lines[4].split(sep)
# [PROCESS THE C VECTOR]
lines[4] = sep.join(c)
# and the atoms:
N_atoms = len(lines)-9
for i in range(0,N_atoms):
    l = lines[8+i].split(sep)
    L3 = float(l[3].strip())
    # [PROCESS THIRD COEFFICIENT]
    l[3] = ' ' + str(L3) + '   '
    lines[8+i] = sep.join(l)
# create the output:
outcontent = '\n'.join(lines)
# now write this to a new FILE:
os.mkdir(outdir+'/' + f)
outfile = open(outdir+'/' + f + '/' + 'POSCAR', 'w')
outfile.write(outcontent)
outfile.close()
```

I then converted the output POSCARs to JSON files for ease of data management (otherwise we would need separate files for every crystal).

⇒ This was achieved by loading the structures in with `pymatgen`, then using the built in `.save()` method to create a JSON file:

```
from pymatgen.core.structure import Structure
folders = os.listdir(outdir)
for i, f in enumerate(folders):
    # load the POSCAR:
    struct = Structure.from_file(outdir+'/' + f + '/' + 'POSCAR')
```

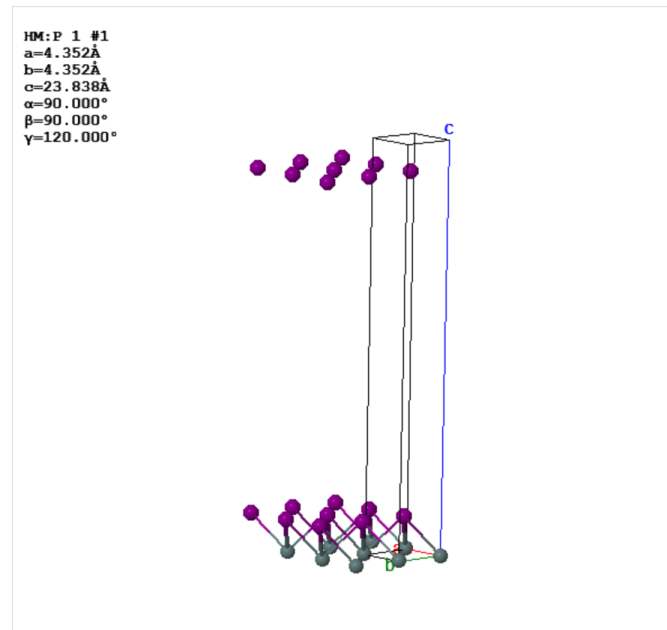
```
# save to a JSON
struct.save(finaldir+'/' + f + '.json')
# remove annoying pickle file
os.remove(finaldir+'/' + f + '.pkl')
```

However, we encountered errors any time we attempted to load one into pymatgen. The error was the worst thing possible in fact, the memory usage would just keep increasing until the program broke.

I later discovered the issue : I had assumed that all of the crystal files had the atoms in a layer right next to one another. However, in quite a few, this wasn't the case:

SnI2

Structure



Taken from <http://mathtc.nsc-c-tj.cn/raman/>.

With everything as normal, these atoms would appear at the bottom of the next layer up, and so each layer would be properly specified. There's no real reason why these aren't at the bottom of the unit cell : they just chose to do it this way.

⇒ Clearly, if we applied our method above to such crystals, the relative positions within the unit cell would be preserved, but the top atoms would **not** be part of individual layers. Rather, they would be isolated between layers, and the crystal structure would be wrong.

If we know the isolated atoms, then we can properly process them by scaling their third coefficient as

$$c_3 \rightarrow \frac{19 + c_3}{20},$$

ensuring that it gets pushed up to the very top, so remains in the right position relative to the layers.

Detecting these isolated points is not trivial, though. I used the following approach:

⇒ Any atom with $c_3 \geq 0.6$ is considered isolated.

Obvious problems will occur if the layer is centred near the middle of the third lattice vector. However, all structure files loaded correctly with pymatgen, so we can assume that this is quite rare (if it occurs at all).

I ran this and saved the output to `adj_jsons/`.

4.2 Day 7 : Tuesday, 21 October

Started by discussing the lattice processing methods I developed over the weekend (see previous section). However, I had run this program for the old (incorrect) dataset, and we had yet to do so for our actual set.

As such, we decided to convert the 2dmatpedia JSON dataset to POSCAR, I wrote a short program to do this (`json_to_poscar.py`). It loads in each file with `Structure.from_file()`, then outputs them as a POSCAR into separate folders (2dm-1/, 2dm-1/, etc.) with `Structure.to()`.

The POSCARs were successfully created, but I noticed that the file structure differed slightly between the outputted format and the Raman database format. I had to make some small adjustments to the program (`datatransfer.py`) in order to function correctly.

We met with our supervisor and discussed next steps for the project. She said that we could begin with the cleaning of the Raman spectra - Ben had programmed a method to obtain the spectra via Gaussian broadening (we are given the eigenvalues and their intensities). To 'clean' the data, we would need to normalise the intensities of each spectrum to the highest peak within that spectrum. Ben started writing a program to do this.

Encountered some problems while attempting to adjust the generated poscars:

- The formatting was slightly different, so I switched from splitting the lines by spaces to splitting the lines by dots.
- This mostly worked, except when python represented the number in scientific notation. As such, I had to format when converting to a string, i.e. `"{: .16f}".format(n)`.

This eventually worked, and I converted the entire 2dmatpedia dataset to a folder of adjusted JSON files (via the methods I discussed earlier), which should be able to be inputted directly into our model.

We trained a band-gap model on the 2dmatpedia dataset, using the CGCNN architecture, which got an MAE of around 0.6 after 29 epochs, and took 2 minutes to train in entirety. This was much quicker than we expected.

The exact command we ran was (in one line):

```
$ python main.py --train-ratio 0.6  
$ --val-ratio 0.2 --test-ratio 0.2 data
```

We saved this trained model as `readj_21-10-25.pth.tar`.

Next, we decided that we would move on to working on the spectrum encoder/decoder. Ben started working on a class to load the spectra as a pytorch dataloader object, while I got to work on the model structure and training algorithm, both in `emb_model.py`. I used the following (simple) structure for the model:

```
class encoder(nn.Module):  
    """  
    Encodes an input raman spectrum via a bottleneck , and then  
    decodes the result.  
    """  
    def __init__( self , spec_length=3000 , encode_length=100):  
        """  
        Initialise the encoder.  
  
        Parameters  
        -----  
  
        spec_length: int
```

```

Length of the input spectrum.
encode_length: int
Length of the encoded vector.
"""

super(encoder , self ). __init__ ()
# Parameters
self.spec_length = spec_length
self.encode_length = encode_length
# Layers
self.encode = nn.Linear( self.spec_length , self.encode_length)
self.decode = nn.Linear( self.encode_length , self.spec_length)
# for now, no hidden layers.

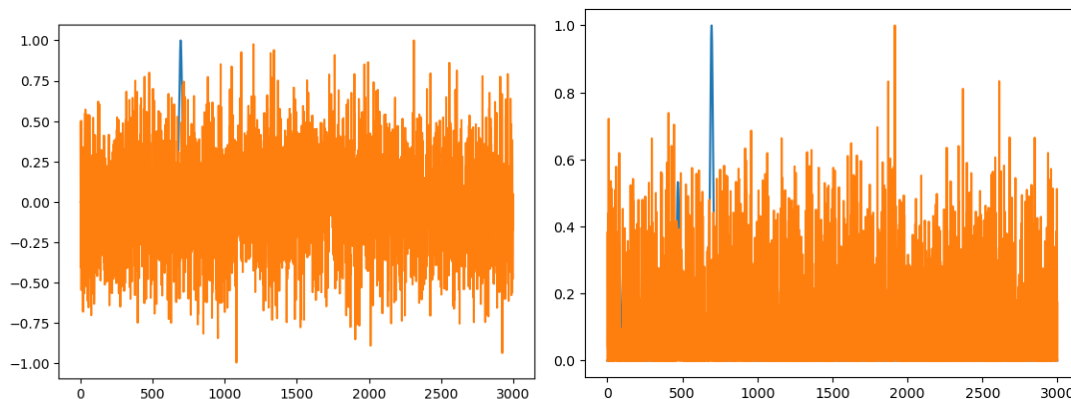
def forward( self , in_spectrum ):
# encoded
enc = self.encode( in_spectrum )
# and now, decode
out = self.decode( enc )
return out

```

For the training code, I followed the pytorch tutorial, <https://docs.pytorch.org/tutorials/beginner/introyt/trainingyt.html>. I made some small changes but the general function is identical.

To get the program to run correctly, I also had to install tensorboard via pip. We ran it on the raman dataset, to train an encoder. After 5 epochs, we achieved a (L1 norm) MAE of 0.073 on the validation set.

We started with a learning rate of 0.001, but we noted that the output was extremely noisy. We added a ReLU activation function after each layer and increased the learning rate to 0.2, which made a **huge** difference to the MAE.



Sample outputs for learning rate 0.001 (left) and 0.2 (right), spectrum length 3000, encoded length 300, 0 hidden layers. Left has no activation function, right has ReLU.

However, the outputs are still clearly nonsense (blue line is what we are attempting to predict).

4.3 Day 8: Thursday, 23 October

Over Wednesday, I worked on attempting to implement a working version of the autoencoder (see `emb_model.py`). I started by modifying our model class, adding a user-specified number of hidden layers. That is, the action would be:

Input $\rightarrow$ Hidden 1 $\rightarrow$ Hidden 2,...,N $\rightarrow$ Encode $\rightarrow$ (ENCODED VECTOR) $\rightarrow$ Decode $\rightarrow$ Hidden 1 $\rightarrow$ Hidden 2,...,N $\rightarrow$ Output

Every layer was followed by an activation function. I trained the model on the Raman spectrum dataset, and found that, with no hidden layers, the predictions were not good at all.

pics

A ReLU activation function seemed to work better than softplus, since most of the values are exactly zero, which ReLU can achieve quite easily.

Note that we generate plots with:

```
f = valid_dataset[0]
g = model(torch.Tensor(f))
plt.plot(g.detach())
```

However, increasing the number of hidden layers did not solve the problem alone; I noticed two main issues:

- The loss function (L1 loss) made it so that the model would tend towards returning very small values almost everywhere.
- Our dataset seemed to have a bias towards low frequencies which meant that, even when I inputted a zero-spectrum, it would still output a peak at the low frequencies.

To solve this second problem, I wrote a program to generate an arbitrarily large dataset of fake spectra...

- ⇒ The input spectra are not 'special'; all the autoencoder needs to do is take some input spectrum, where the form is mostly zero, with a few gaussian peaks, and encode this.
- ⇒ My program randomly generates 2-8 frequency 'eigenvalues', with intensities between 0 and 1, then convolves each with a gaussian of std 25 indices over a domain of 3000 indices.
- ⇒ To ensure that the intensity is well-defined, I check to make sure that eigenvalues are separated by at least 25 indices. If they aren't, I resample.
- ⇒ This is very quick, and I used it to make a dataset of 12000 spectra.

After training a model on this, I found that it has NO bias, which is good. However, all of the values tend to zero after enough training - this seems to be the fault of the loss function, which does not penalise zero-values at peaks enough.

plots?

I found a post from someone with a similar problem who suggested using a 'weighted loss function' (<https://stackoverflow.com/questions/57618482/higher-loss-penalty-for-true-non-zero-predictions>) and I tried to implement this as a custom loss function.

- ⇒ supposedly we can just def lossfunction: ..., given that the mathematical operations are performed with the torch library.
- ⇒ this seemed to work for training, but went haywire for the validation dataset; more consideration required?

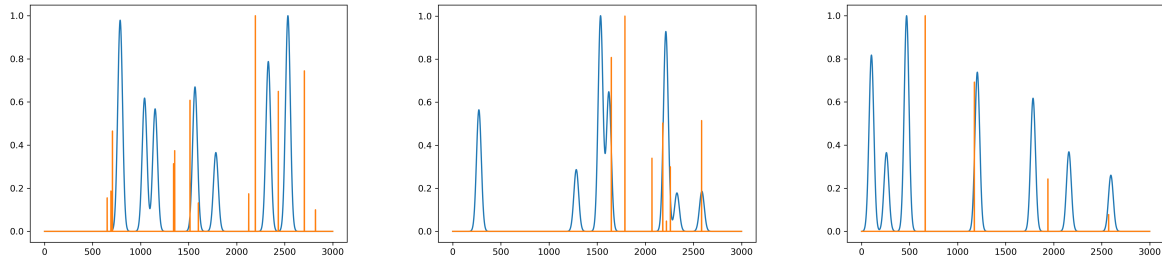
Additional observations:

- Most people seem to use a learning rate of 0.01. This only seems to work for a large number of epochs; for <20 epochs, I found that 0.1 is sufficient.
- The length of the hidden vectors was set to be 1000 for these examples, but this is fully customisable in my program.
- We used a split of 0.6 (training), 0.2 (validation), 0.2 (unused).

- My kernel is weird, so unless I import matplotlib before all of the other packages, the kernel crashes whenever I try to make a plot.

I did a run with the full artificial dataset, with 2 hidden layers (1000 length vectors), and a learning rate of 0.1 (using L1Loss). The results were much better:

- We got thin peaks, which seemed to correspond well to our inputs. There was no observable bias in results, and most of the values were zero (as we would hope).



Note: prediction values have been normalised.

- Overall, this was much better, though there were still a few issues:
- The magnitude was extremely small ($\sim 10^{-3}$)
- There were a lot of 'made up' peaks, although they were usually concentrated around the actual peaks.
- The peaks were extremely thin (using softplus might help with this?)

However, I realised an issue with my dataset, in that most of the spectra were not properly normalised (the maximum value being somewhere between 0.1 and 1.0). This was easily fixed in the spectrum generator with `ints = ints/np.max(ints)`, and I generated a new dataset of the same size (12440) to run a new model.

We thought that adding a convolutional layer at the start might improve the model's ability to recognise covariance between adjacent points, but we struggled to implement a class which worked as we wanted. Specifically, we wanted to sum over adjacent points, but couldn't think of an efficient way to do this.

For now, I reran the training on the new dataset with the following hyperparameters:

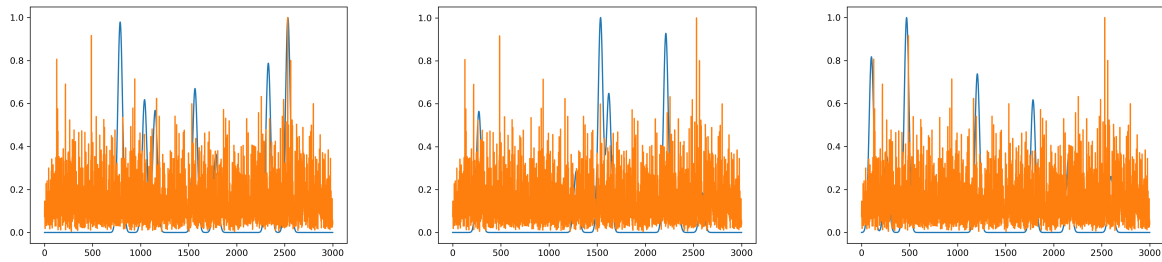
```
p_train = 0.6
p_val = 0.2
n_hid = 2
h_len = 1000
learnrate = 0.1
EPOCHS = 12
```

This produced similar results (plots essentially identical to above), I made a few notes:

- It seems very good at counting the number of peaks, and placing them in an approximate location.
- It is very bad at placing peaks at exactly the right positions relative to the actual spectra.
- The magnitudes are still very small ($\sim 10^{-3}$)

Ben ran a model with 4 hidden layers, and the same hyperparameters as before. We didn't see improvement in the model's performance, unfortunately.

At the same time, I ran a model with 4 hidden layers, and a softplus activation function - this did even worse, as the output was extremely inflexible, and the results was just noise. It seems like ReLU is a much better activation function, overall.



Softplus results (specifications same as above besides 4 hidden layers).

We thought that the loss function might be causing the lack of good results - we proposed two possible improvements:

- (a) Multiplying the L1 loss by a large factor if the output was below a threshold, e.g. 0.05
- (b) Multiplying the L1 loss by a large factor in areas where the target was large (i.e. penalising wrong values around peaks)

We implemented these as custom loss function in `emb_model.py`, where we defined a function with inputs (output, target), and which modified the tensors via pytorch functions. We noticed:

- (a) led to a model which always outputted 0, unfortunately.
- (b) led to a large amount of noise.

plots?

Both seemed to make very little progress on reducing their losses as they trained - we wondered why it is possible that the L1loss works so much better when it is (in principle) very similar?

I decided to run a quick test to see if a custom-defined L1 loss would achieve the same results as the built-in L1loss function. It did (albeit slower), so the problem isn't solely that our functions are custom.

⇒ I read online that SGD will only work if the loss function has a well-defined derivative, something to consider? (<https://stackoverflow.com/questions/56839201/my-custom-loss-function-in-pytorch-does-not-train>)

5 Week 5 (INCOMPLETE!!!)

5.1 Day 9 : Tuesday, 28 October

We received an email from our supervisor which suggested that we give up on the spectrum embedding, and instead, move on to directly predicting the excitation eigenvalues. This should simplify the problem greatly, since we no longer need to write an embedding program for the spectra, and instead can simply use a list of eigenvalues as the output of our model.

We decided that we could directly use the CGCNN structure, but add a few more hidden layers, and change the output length to match our list of eigenvalues. For the output of this model, we decided that it would be a 4-length vector, where components corresponded to the 'top 4' eigenvalues, in terms of intensities.

I got to working on the model structure/ training program. We had to change the original structure, since now the target properties were vectors. I decided to write the training in my own format, which I've stored in `eigenvalue_model/eig_main.py`, and made small adjustments to the CGCNN structure, which is in `eigenvalue_model/eig_model.py`.

Writing the program was essentially the same as for the spectrum embedder previously, but with one notable difference...

- In our previous model, we had neglected to normalise the data - apparently this is quite important.
- The CGCNN model takes a random sample of the dataset, then computes the mean and std of the sample targets, and uses this to normalise target values with $(\text{targ} - \text{mean})/\text{std}$.
- For a vector of target values, this is less clear..
 - ⇒ for simplicity, I just divided each target by a constant scaling factor, which was equal to the maximum frequency of the entire dataset (4410)
 - ⇒ this should normalise the targets between 0 and 1

We had to make a few changes to JSONLoader to accommodate the vector target values; for some reason we kept encountering an issue where the name of every single crystal would be printed on the first epoch.

I tried to write my own functions for the validation and training loaders, but since the structure of their data is so strange, I had to just use their function (`get_train_val_test_loader`), which seemed to work fine.

We were able to successfully train a model on the Raman dataset, where we used `MSELoss` for the loss function, and calculated the MAE with the denormalised targets and outputs. The first run:

```
n_h = 10
n_conv = 3
h_fea_len = 128
atom_fea_len = 64
EPOCHS = 30
```

⇒ MAE: 254.21

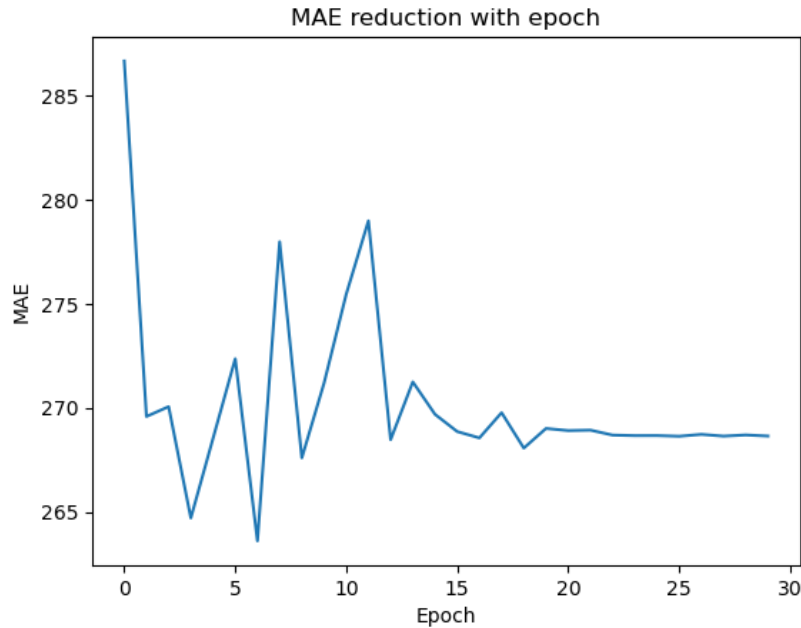
Observations:

- Model trained very quickly, which is good.
- We think that setting 'None' eigenvalues to zero might be introducing some bias into the system... I suggested making these values uniformly sample over the frequency range...

READ BEN'S LAB BOOK TO FINISH UP DAY 1

5.2 Day 10 : Thursday, 30 October

Ben was able to achieve pretty good MAEs over the days between, getting one as low as 87 - however, this was only for one dataset; it seemed that most of our runs converged around ~ 250 , which is not good at all.



MAE reduction for a typical model.

We thought that this might be due to poor hyperparameter choices, so we decided that we might do a few runs over different hyperparameters and see if we could identify some optimal choice. I imagined that the first step to achieving this would be compressing the training of a model into a single function; to this end, I began writing the program `eig_hyps.py`, for which I made a few notable changes from `eig_main.py`:

- The training and validation of the model occurs in a function 'trainModel()', which takes parameters:

```
def trainModel(  
    lr=0.001,  
    EPOCHS=30,  
    p_train = 0.6,  
    p_val = 0.2,  
    p_test = 0.1,  
    seed=123,  
    n_h = 25,  
    n_conv = 3,  
    h_fea_len = 128,  
    atom_fea_len = 64,  
    root_dir = "C:/Users/bjama/Desktop/big_NN/big_NN/data",  
    adam = False,  
    infin = False  
)
```

- This contains all of the action of `eig_main`, with some subtractions:

- ⇒ All printing is removed.
- ⇒ Model saving is removed.
- ⇒ CUDA is not available.

- I had noticed in the CGCNN training program, 'main.py', that they had attempted to implement a variable learning rate. This seemed like a good idea, since we would want the model to converge quickly around a minimum, and spend the rest of its time achieving the optimum value.

⇒ I found an improvement over their program: pytorch has a built-in set of learning rate 'schedulers', in `torch.optim.lr_scheduler`, which will do all of the work for us.

⇒ The `ReduceLROnPlateau()` one seemed like it would be a simple improvement, since this reduces the learning rate if changes in some value are less than a specified threshold for more than some specified number of epochs (the 'patience'). I added one as:

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, "min", patience=5)
```

⇒ The default decrease is a factor of 10 per activation. The scheduler automatically works if we put `scheduler.step(avg_vloss)` after the validation, which is exactly what I did.

⇒ If the scheduler makes the learning rate too small, then I want to end the program, so I added an if statement to account for this possibility, where I set `thres=10-4`:

```
if infin and (scheduler.get_last_lr()[0] <= thres):
    print("Ending due to small learning rate.")
    break
```

⇒ `infin` is an optional boolean parameter to the function (default is false) which turns on an 'infinite epoch mode', where the evaluation continues until the scheduler decreases the learning rate below our threshold. This is quite useful for achieving the 'absolute minimum' for a given model.

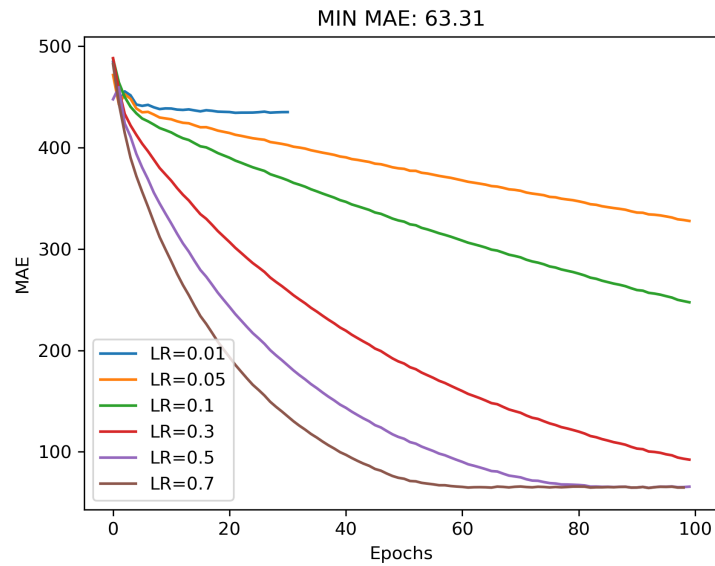
- One paper I read mentioned that they used the 'adam' optimiser instead of SGD, and I tried to implement this, but found that it really didn't work at all. I'm not sure what went wrong but the remnants are still in the program in the form of 'adam=False'. It should always be false.
- The `trainModel()` function returns two numpy arrays, one containing the validation MAE at each epoch, and the other containing the validation loss at each epoch. They are retrieved as:

```
mae_array, loss_array = trainModel(...)
```

- Additionally, added a line to make results reproducible, `torch.manual_seed(seed)`, where `seed` is an input to the function.
- Besides this, the action is identical to `eig_main`, and everything works well.

I then decided to test a few learning rates with this convenient function, and ran a test for 0.01, 0.05, 0.1, 0.3, 0.5, 0.7, each with the scheduler enabled. We made a few observations:

- We had initially thought that a small learning rate would be best, but we actually found that large initial learning rates performed significantly better.



Good results? Not all is as it seems...

- However, we had noticed that the loss was very small throughout the training and validation - we realised that our normalisation was incorrect! This is a big issue...
 - ⇒ We had not changed the normalizer since when we were just training on 4 eigenvalues, so it was *also* dividing the intensity values by 4000, meaning that the model was likely just making them zero, and that their contribution to the loss function was practically zero. This was an easy fix.

6 Week 6 (INCOMPLETE!!)

6.1 Monday, 3 November

My tests have shown that the model is outputting a constant result for all inputs...?

I have a few initial theories as to what could be causing this:

- (a) An error in passing the data to the model.
- (b) An error in loading the data.
- (c) An error due to the model itself.

I wrote a function to test a model on some elements of the validation set [testmodel() in 'eig_hyps.py'], which works for any CGCNN-structured model, since the weights/biases are inputted.

(a) seemed the most likely... I read a few stack exchange posts on this:

- The most common statement I saw was that the use of 'Variable(...)' for dictating tensors that should be gradient-tracked is deprecated.

⇒ I doubted that this was really the issue, but just to be safe I updated all of the places where Variable is called to a tensor-oriented form, i.e.

```
var = Variable(something) -> var = something.requires_grad_(True)
```

given that 'something' is a tensor. Otherwise we would directly define a new tensor with torch.tensor(...,requires_grad=True), but this is a case which didn't come up. Note the use of lowercase 'tensor'.

- Many other people seem to have had similar issues to me:
 - (<https://discuss.pytorch.org/t/help-understanding-model-output/27152>)
 - (<https://discuss.pytorch.org/t/models-give-same-output-despite-having-different-input/183986>), they also use CGCNN!
 - (<https://stackoverflow.com/questions/75800779/i-get-the-same-prediction-for-different-inputs-pytorch-neural-network-regressio>)

⇒ Some mentioned that batch size might be the problem, as in when the batch size is too large, the model fits itself simply to the average of the batch, which is close to the average of the data.

⇒ We have been using 256 for the batch size, so I tried training with 32, but I saw no real improvement, and the same issue persisted.

- (https://www.reddit.com/r/pytorch/comments/15z7vzd/model_output_for_test_dataset_is_always_the_same/) suggests that lowering the learning rate might make a difference. However, this did not seem to help.

I figured that if the source of the problem occurred during training (a set of weights going to zero, somehow), then putting a set of values into an entirely untrained model (i.e. weights completely randomised) would spit something different out for each input.

⇒ HOWEVER, when I tried this with an initialised CGCNN, it had the same problem! This allowed me to eliminate training as the source of the issue.

My best guess for the problem would then be something in (c), since I was relatively certain that there were no issues with the data loading. It occurred to me that the size of the model might be playing a factor, since a huge number of hidden layers would possibly mean that the input values were lost somewhere in the middle...

- I had previously tried inputting random values to a single linear layer, and this indeed returned a range of outputs. Clearly, there was no problem with the underlying package.
- We had been using (at least) 25 hidden layers for the past week or so, so I decided to put this down to a 3 hidden layers - and **suddenly**, the outputs were no longer identical!

OUTCOME: The problem of fixed outputs is due to an overly large number of linear hidden layers.

Note: the following is for seed 777.

I then proceeded to train a model with *one* hidden layer, and a hidden feature length of 64 (we had been using 128 previously), and this turned out to be my best model yet.

- After 70 epochs, it converged on an MAE for the first frequency of 160 (most previous models had been giving upwards of 240). I also observed that the loss steadily declined throughout the training, and was even going down close to the 70 epoch mark - previously, models had just converged on the 'best' outcome almost instantly.

⇒ This could indicate better performance in capturing the complex mapping!

I later achieved an even better fit with 1 hidden layer, with a hidden feature length of 100, and a scaling of 500.

⇒ The best MAEs per frequency here were $\sim (146, 163, 162, 173)$, and the lowest achieved loss was 0.136 after 100 epochs.

Trying with 150 features, the performed suffered a slight drop, with the best validation going as low as 0.138.

⇒ However, when I increased the initial learning rate to 0.01, this model suddenly became significantly better! We achieved a lowest validation loss of 0.126!!

⇒ The best MAEs per eigenfrequency were $\sim (133, 152, 148, 151)$

⇒ Increasing the LR to 0.1 gave further improvements, as the min loss fell to 0.120 in 50 epochs, with best MAEs $\sim (121, 144, 142, 151)$

⇒ Further increasing LR to 0.4 improved the min loss to 0.117, with best MAEs $\sim (116, 144, 138, 150)$ after 60 epochs.

This outcome was a surprise to me; I had thought that adding more hidden layers would uniformly lead to improvements in the performance of the model, but this is clearly not the case.

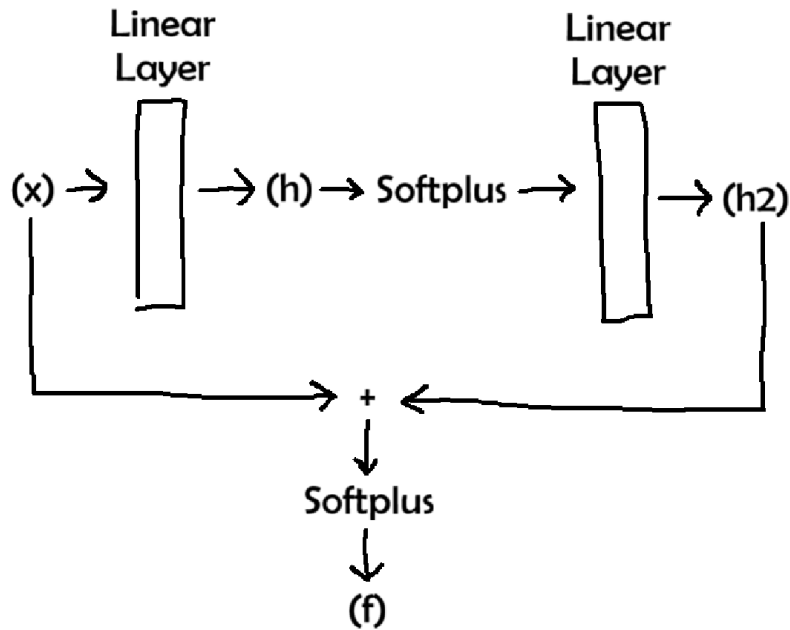
⇒ We will need to be more creative with our structure when it comes to extracting more performance from the model.

Things to consider for tomorrow:

- ResNet models; try to avoid vanishing gradients by letting previous outputs 'skip' layers. I already implemented this, but it can be improved.
- RNNs/LSTM; easy to implement since they are baked into PyTorch.
- Adversarial network? Once model tries to tell if eigenvalues fit a lattice, other model tries to generate eigenvalues?

6.2 Day 11 : Tuesday, 4 November

I decided to do a few runs with some ResNet layers I implemented last night. The structure of these is very simple:



Resnet structure. Just a simple version.

That is, the input to the 'block' jumps to the end and the result,

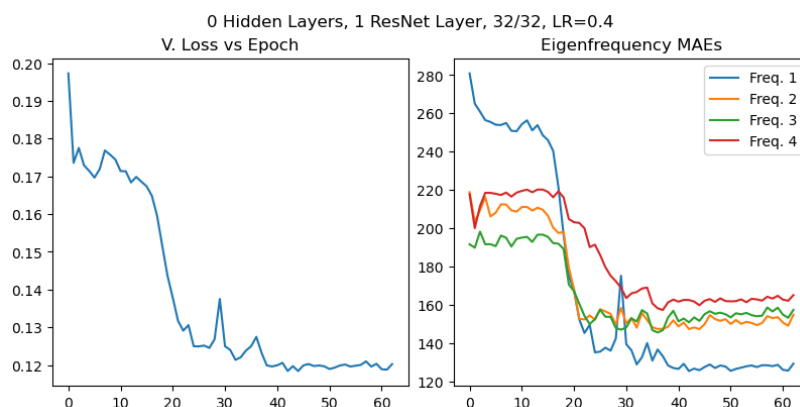
$$H(x) = F(x) + x,$$

where $F(x) \equiv H(x) - x$ is the *residual function*, i.e. some combination of layers. They should in practice be more complicated than the above.

⇒ (x) needs to have the same dimension as (h2), but (h/h1) can be whatever we want. Typically, this may be a bottleneck, so I'll denote the feature lengths of such a layer as, e.g. 64/32, indicating an input/output of 64 and bottleneck of 32.

Even with a single residual layer, the results didn't seem to be great (certainly not an improvement on the single linear layer we had in previous iterations).

- I was able to get a good result for a single 32/32 layer, with an initial learning rate of 0.4. There wasn't a huge improvement over previous models, but it clearly indicated that poor models were converging on a local minimum (close to the starting weights).



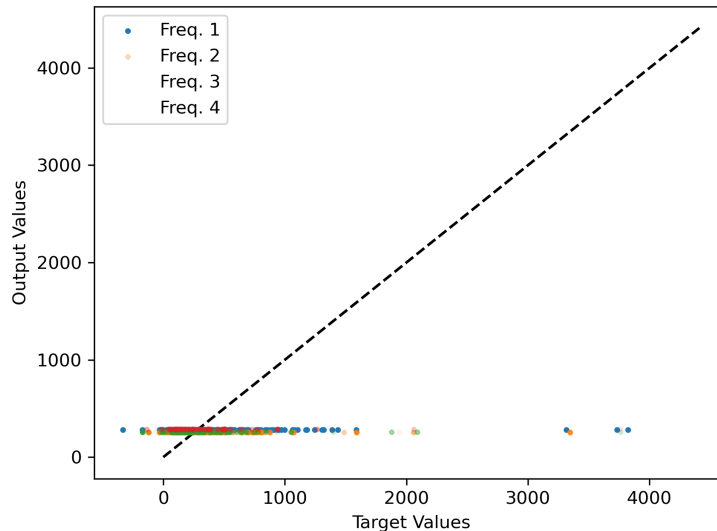
'Local minimum' convergence is clear.

⇒ Indeed, this model had initially converged on this minimum, but was able to 'jitter' out thanks to the high learning rate. The final result was decent, min loss just under 0.12, and maes ~ (125, 147, 145, 157). Not our best, but **much** better than what I had been seeing before.

I noticed that models in this local minimum tended to have a few distinct features:

- Constant outputs
- First intensity 1, all others almost zero.
- Highest MAE in the first frequency, lowest in the last (contrary to what we would expect)
- No overfitting.

An output-target plot illustrates exactly the problem with such models:



Constant outputs are clear. We would hope for the outputs to be along the line of equality.

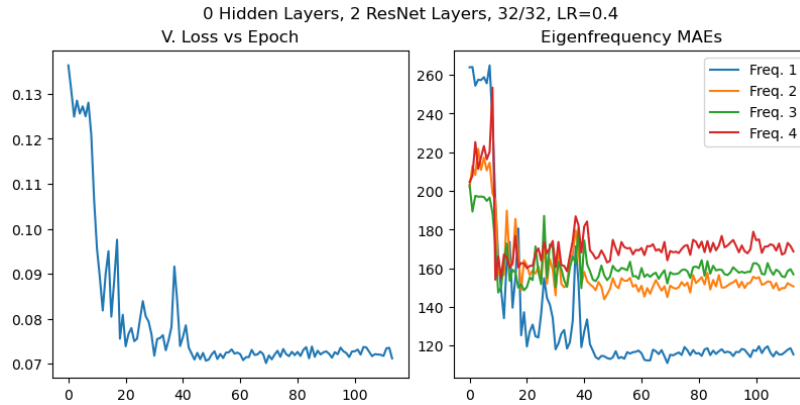
My running theory is that these correspond to minimisation of the 'intensity' part of the loss function, while the frequency part is essentially ignored (i.e. goes to the average).

⇒ This would make sense, since our target normalisation ensures that most frequencies are between 0 and 1, and the frequency difference is certainly less than ~ 0.2 in a working model, which is generally comparable to the difference in intensity, since I have been using $k_1 = 1$, $k_2 = 0.8$.

As such, I think that to make progress, we need to:

- (a) Decrease k_2 significantly w.r.t. k_1
 - (b) Attempt to have a high variability in our SGD, to escape the local minimum (if it still exists).
- ⇒ Achieving (a) is simple, I will test a few values of $k_1 = 1$, $k_2 =$ (e.g. 0.2, 0.1, 0.01), and see if lower k_2 helps to more consistently achieve the local minimum. Then I will see if this allows us to scale up the model.
- ⇒ (b) is also relatively simple to achieve; I read here (https://www.reddit.com/r/deeplearning/comments/fe0mrc/which_hyper_parameters_to_change_to_avoid_getting/) that the following should help:
- Increasing base learning rate (already done)
 - Decreasing batch size

I tried to run another model, with 2 resnet layers (previously had not converged), $k_1 = 1$ and $k_2 = 0.2$, and a split of 32/32. We got a good result!



⇒ This gave us our best first-frequency MAE yet (as you would expect if the model was made more complicated), $\sim (110, 143, 147, 154)$, with min loss 0.07 (but this is not comparable to previous results since the scale is different).

⇒ I checked directly, and the outputs indeed have a high degree of variability, attempting to match the target rather than simply averaging out. Very good!

⇒ Reducing to $k_2 = 0.01$ allowed me to train even 3 resnet layers, though the output was not greatly improved over previous models, with best MAEs $\sim (118, 145, 141, 152)$. Still - it is good to know that it is possible to train more complicated models properly.

⇒ I confirmed that the outputs varied properly (prepare for long text):

Testing index 0:

Normed output:

0.6978, 0.6326, 0.5971, 0.5728, 1.0031, 0.4383, 0.1906, 0.0608

Denormed Output:

3.4891e+02, 3.1632e+02, 2.9854e+02, 2.8639e+02, 1.0031e+00, 4.3830e-01, 1.9062e-01, 6.0831e-02

Target:

3.5707e+02, 5.1090e+01, 1.7640e+02, 1.6941e+02, 1.0000e+00, 3.2764e-01, 1.4948e-01, 2.4626e-02

Testing index 1:

Normed output:

0.7673, 0.6949, 0.6490, 0.6212, 1.0046, 0.4519, 0.1978, 0.0684

Denormed Output:

3.8366e+02, 3.4744e+02, 3.2448e+02, 3.1058e+02, 1.0046e+00, 4.5190e-01, 1.9779e-01, 6.8412e-02

Target:

1.3244e+03, 1.3398e+02, 1.1335e+02, 1.9371e+02, 1.0000e+00, 1.2313e-01, 1.1249e-01, 5.1817e-02

Testing index 2:

Normed output:

0.5116, 0.4664, 0.4554, 0.4393, 0.9986, 0.4001, 0.1712, 0.0375

Denormed Output:

2.5580e+02, 2.3318e+02, 2.2769e+02, 2.1965e+02, 9.9860e-01, 4.0013e-01, 1.7116e-01, 3.7512e-02

Target:

6.4701e+02, 4.8483e+02, 3.8096e+02, 1.7038e+02, 1.0000e+00, 5.5725e-01, 3.1034e-01, 2.2105e-01

Testing index 3:

Normed output:

0.7869, 0.7115, 0.6626, 0.6342, 1.0049, 0.4544, 0.1990, 0.0714

Denormed Output:

3.9347e+02, 3.5576e+02, 3.3128e+02, 3.1712e+02, 1.0049e+00, 4.5441e-01,
1.9897e-01, 7.1397e-02

Target:

4.3931e+02, 1.9164e+02, 2.9676e+02, 4.0424e+02, 1.0000e+00, 2.9954e-01,
2.0594e-01, 1.0271e-01

Testing index 4:

Normed output:

0.2197, 0.1783, 0.1955, 0.1917, 0.9882, 0.2905, 0.1175, 0.0151

Denormed Output:

1.0987e+02, 8.9155e+01, 9.7763e+01, 9.5872e+01, 9.8819e-01, 2.9046e-01,
1.1752e-01, 1.5108e-02

Target:

1.0077e+02, 1.5845e+02, 5.7036e+01, 5.7053e+01, 1.0000e+00, 2.9217e-01,
6.6720e-02, 6.6706e-02

Testing index 5:

Normed output:

0.8070, 0.7289, 0.6773, 0.6477, 1.0053, 0.4578, 0.2009, 0.0737

Denormed Output:

4.0349e+02, 3.6443e+02, 3.3863e+02, 3.2384e+02, 1.0053e+00, 4.5784e-01,
2.0089e-01, 7.3655e-02

Target:

3.8590e+02, 2.8996e+02, 2.9035e+02, 2.7272e+02, 1.0000e+00, 7.4953e-01,
7.0078e-01, 2.7577e-01

⇒ This sort of result is exactly what we want. It seems like we can keep reducing k_2 and increase the complexity of the model to achieve better results - but surely there is a limit...?

We decided to do a short literature review, I found this paper, which seemed interesting.

https://pubs.acs.org/doi/pdf/10.1021/acs.jctc.4c01151?ref=article_openPDF

- DFT error propagates into ML models. Problematic? Probably not for us.
- Proposes a GNN for predicting a wide range of properties. "xPaiNN"
 - ⇒ Builds on previous work (<https://arxiv.org/pdf/2102.03150>)

We specifically liked the ideas discussed in Sorrentino et. al (MOL2RAMAN)...

(<https://chemrxiv.org/engage/chemrxiv/article-details/68263e1f50018ac7c5659c45>)

- They employed a transfer learning approach, where a model was first trained on predicting the number of Raman eigenvalues based on a molecular structure, and then the output of this model was fed to another model, which predicted the spectra directly.
- Their GNN structure was also slightly different from the CGCNN, though we noticed that the fundamental steps were almost identical (~ 4 'GINEConv' layers which are pooled together, then passed to a sequence of hidden layers).
- In particular, their convolution pooling step was slightly different : they concatenated the outputs of each convolution layers rather than taking only the final one (which was what the CGCNN did).
 - ⇒ Since this was relatively simple to implement, I made a new class 'cat_CGCNN', which used this new pooling method.
 - ⇒ Did not notice huge performance gains. Likely better for larger models, but for now the effects were negligible. Though there is no reason not to do it.

We decided to attempt to make a model for predicting the number of peaks of a crystal, like this paper did. Since it seemed like their approach was just to add 1 hidden layer on top of their crystal embedding program, we figured that we might be able to achieve similar results with 1 hidden layer on top of the CGCNN.

- ⇒ This gave incredibly bad results; we were only able to achieve a best MAE of 2.5, and it seemed like the outputs of all of the models we tried were just constant regardless.
- ⇒ The file containing the numbers of peaks is 'id_prop_num_peaks.npy', and we had to make a number of small adjustments to the data loader and 'eig_hyp.py'

6.3 Day 12 : Thursday, 6 November

Had a meeting with our supervisor, discussed next steps. She suggested:

- Modifying our loss function to see if results are improved.
- Try some sort of spectra prediction with reduction algorithm from Sorrentino et al.
- Start taking data for the report!

We decided to try to program a model to predict the first frequency only, since we would "expect" that this would give the most accurate possible result. This was easy to achieve, since we could use the program we wrote for the number of peaks, and just generate a new id_prop.npy.

- ⇒ Initial results suggested that this was becoming stuck in the local minimum - I was surprised by this, since given that there was no intensity contribution, I didn't think that the local minimum would exist...
- ⇒ MAEs of ~ 250 . Trained with L1Loss.
- ⇒ We were unable to get this to converge, so we just decided to abandon the idea for now, since there's no point really wasting time on it if nothing will happen.
- ⇒ I still don't really know why our larger model is able to converge to a variable solution while this one gets stuck.

We decided...

FINISH BY LOOKING AT OTHER LAB BOOK!!

7 Week 7 (check it!!)

7.1 Day 13 : Tuesday, 11 November

We laid out our next major step for this project...

- ⇒ Since it seems like this current version of the model isn't making significant improvements, we will do a BIG iteration over hyperparameters to try to obtain the best possible version, which will be what we write about in our report.
- ⇒ We decided to iterate over the following hyperparameters:
 - n_h from 0,1,2,3,4
 - n_{res} from 0,1,2,3
 - hidden feature length from 50,90,150,250,350
 - 6 repeats on different seeds for each combination, to get the best model.

Additionally, we wanted to test some different loss functions as an extra degree of freedom; we decided to use the following in this comparison:

1. Our original weighted loss function.
2. Intensity-corrected MAE (i.e. removing nonsense and intensity component)
3. Sorting weighted loss function
 - ⇒ Sorts output and target from low to high frequency then does the same operation as (1).
 - ⇒ We think that this might perform better due to removing the order restriction.

We had a discussion over whether the final loss (3) would perform similarly to just sorting the target values from low to high (in frequency, then moving the intensities on the right to the correct positions).

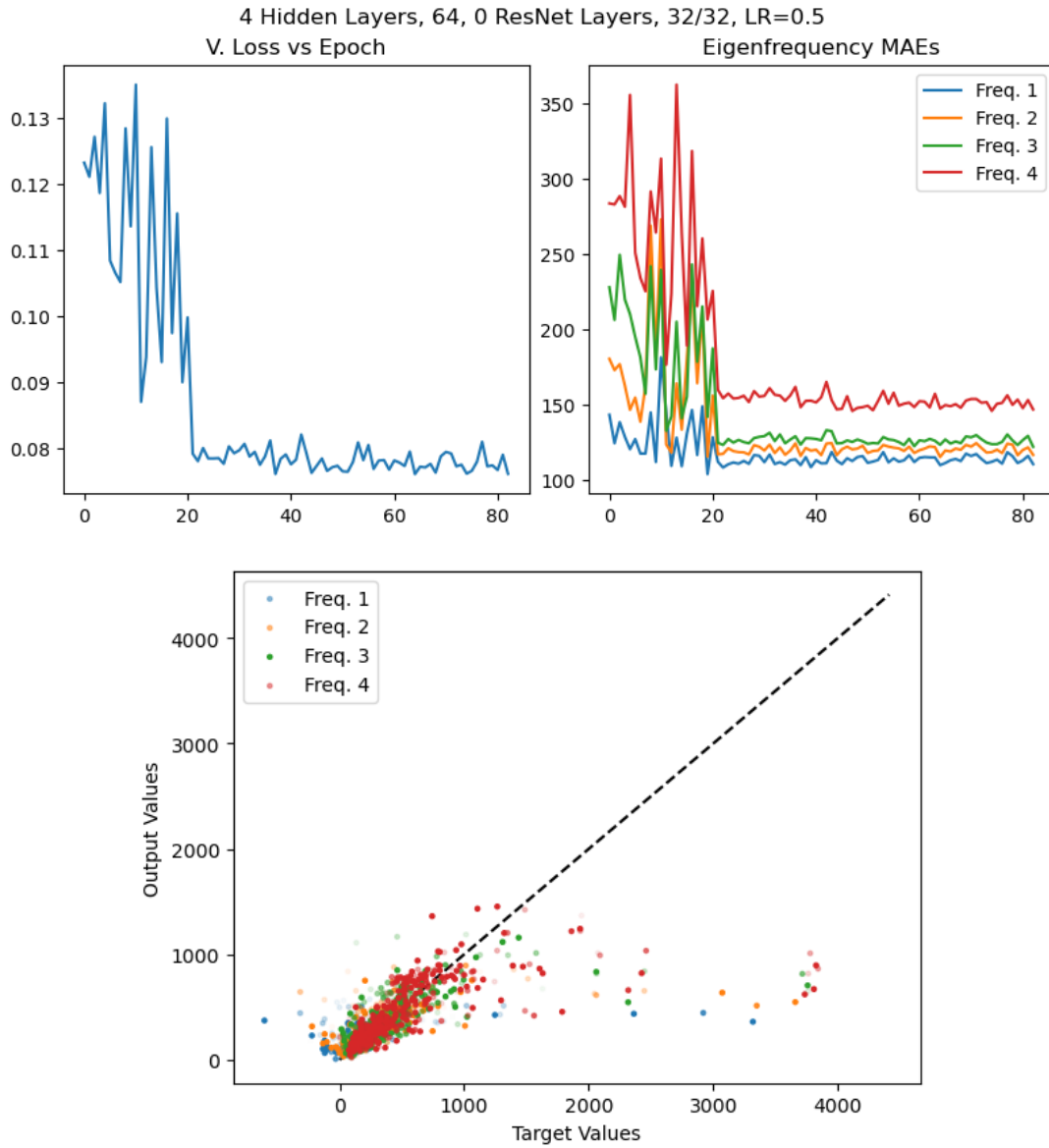
- ⇒ I thought that choosing this target structure, and using our original loss (1) would improve the performance.
- ⇒ This is because it brings us closer to a bin-based model; the first output will usually lie between 0 and 100 (the smallest frequency), while the others are similarly stratified. I thought that this would allow each to train more accurately, since its frequency range was restricted, rather than being the entire possible range of frequencies.

This was easy to program; I just loaded in the previous 'id_prop.npy', sorted the first halves of the numpy arrays with np.argsort(), then propagated this sorting to the final four indices by adding 4 to each of its values.

- ⇒ This program is saved as 'idpropsorter.py' in /big_NN/
- ⇒ The output file was saved as 'id_prop_sorted.npy' in /big_NN/data/

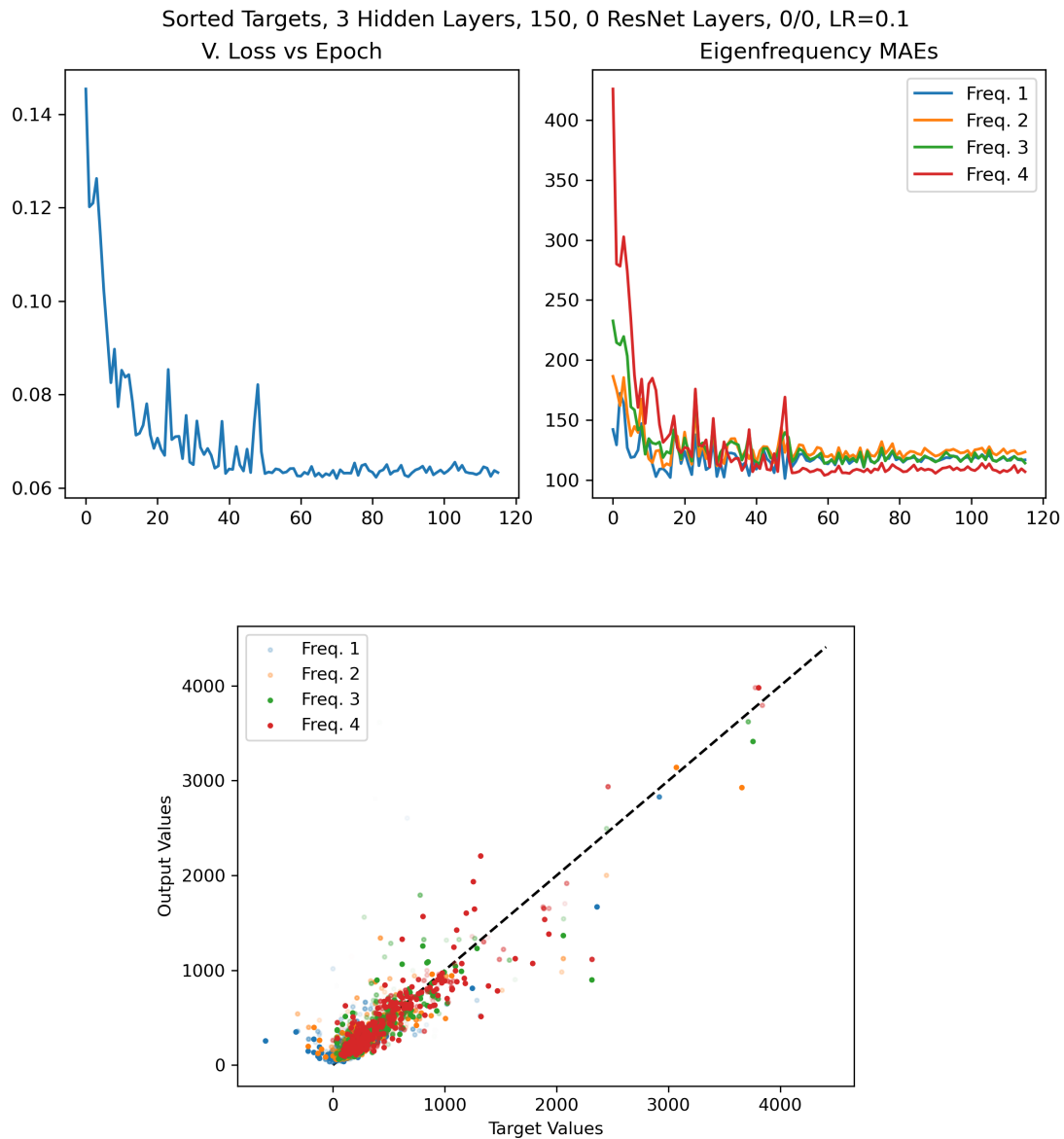
I decided to train a few models with this new target structure, and started with $n_h = 2$, with feature length 150; $k_1 = 1$, $k_2 = 0.1$; LR=0.1, scale=500; testing=True, and seed 456.

- ⇒ This gave pretty good results. We converged on loss=0.0762 after 82 epochs, and the best MAEs were $\sim (104, 115, 122, 145)$, which was the best overall performance I had seen to date.
- ⇒ However, a plot of outputs vs targets showed that it performed poorly for high target values, and each output struggled to predict anything outside of a small given range.



After training this model, I realised that momentum had been set to zero, so I decided to train another model, with $n_h = 3$, $h_fea_len=150$, and $mom=0.9$ (all else the same).

- ⇒ This had GREATLY improved performance! We converged to a minimum loss of 0.062 after 115 epochs, with minimum MAEs *sim*(101, 110, 110, 103). This was by far the best performance I had seen from a model.
- ⇒ Furthermore, the output-target plot showed great improvement over the previous version, where previously constant outputs at high frequency had now very clearly started to follow the line of equality.



⇒ Interestingly, the first and final outputs had the lowest MAEs, while the middle two outputs were somewhat higher. This can be explained by the uniform sampling of non-eigenvalues, I thought.

I was greatly enthused by these results, since with just a small change to the data formatting, we had unlocked a massive amount of performance - if I corrected the MAE for our randomly chosen values, it's very likely that these would all be 80, which is a huge huge boost in performance.

It's a bit silly that I don't exclude the nonsense values from my MAE, so I decided I would just program this in quickly.

- Surprisingly this did not reduce the MAEs by much. The same model outputted minimum MAEs $\sim (99, 110, 111, 102)$. BUT we finally broke the 100 MAE mark!!

7.2 Day 14 : Thursday, 13 November

We noticed yesterday that there was an issue with `id_prop.npy` - the sorting was wrong for some edge cases; Ben noticed this so I can't say too much about it (see his lab book).

⇒ I decided that I would write a program myself to generate the `id_prop` files, in as simple of an implementation as I could, since Ben's file had become very complicated.

⇒ This file is '`idpropmyself.npy`', and I generated 'fixed' versions of each `id_prop` file.

Our overall goal for the day was to run our huge hyperparameter iteration on the office computer (which

has a powerful GPU). We received the login details from our supervisor, the address was yihao@10.206.79.67, and the password is in my email (I won't put it here).

⇒ I had to first upload our data files to the SSH server, putting them in the directory /home/yihao/mphys_bens.

Here is the process I follow:

1. login to ssh with `$ ssh yihao@10.206.79.67`, enter password when prompted.

2. cd to /home/yihao/mphys_bens, pwd to get directory string.

`$ mkdir data.`

3. In another powershell instance, cd to big_NN/data

4. Run:

```
$ scp -r . yihao@10.206.79.67:/home/yihao/mphys_bens/data
```

This recursively copies files from the current folder.

5. For other files, run:

```
$ scp <file -name> yihao@10.206.79.67:/home/yihao/mphys_bens
```

Ben created a local conda environment in the folder, and installed packages with:

```
$ conda create -n CondaB python=3.12 -y
$ conda activate CondaB
$ conda install -n CondaB <package -name>
```

For pytorch, we had to run the following install command:

```
$ conda install -n CondaB pytorch pytorch-cuda=12.1 -c pytorch -c nvidia
```

We had to install the following:

- PyTorch
- pymatgen
- matplotlib
- numpy

Strangely, we noticed that pymatgen itself installed python 3.12 - we assume that this will not cause any issues.

8 Week 8 (INCOMPLETE!)

8.1 Monday, 17 November

We noticed continuing issues with our SLURM jobs - namely, the computation seemed to stop after finishing the first iteration. We weren't sure how to fix this, but the calculations were quick.

Ben had written a version of the program to run on GPU, which we thought might be quicker. I set up a working GPU environment (CondaB3) in the SSH server with the following commands:

```
$ conda create -n CondaB3 python=3.12
$ conda install -n CondaB3 conda-forge::pytorch-gpu
$ conda install -n CondaB3 conda-forge::matplotlib
$ conda install -n CondaB3 conda-forge::pymatgen
$ conda install -n CondaB3 conda-forge::tensorboard
```

We were able to run the GPU version overnight with the following command:

```
$ nohup [python.exe path] [program path] > resnet_log.txt 2>&1 $
```

That is, it runs the [program] on our specified [python.exe], and saves the command line outputs to resnet_log.txt, within our current working directory.

⇒ For that, we should ensure that we are in mphys_bens/ before running this.

We had a few errors, and it seemed like we had made some trivial errors, like not deleting the directories for previous runs (which would cause the program to break).

⇒ Decided that we would come back to this on Tuesday.

8.2 Day 15 : Tuesday, 18 November

We decided that we would attempt to fix the problem with the CPU version of our hyperparameter iteration.

We had additionally noticed that it had seemed to do no computation overnight, and suddenly resumed once we logged on in the morning.

⇒ After narrowing down where the stall was occurring, we found that it must occur in the plotting at the end of each 2D hyperparameter iteration.

⇒ After some searching, we found that the line plt.show() MUST be causing the issue...

- It opens a window with the plot, but the program is HALTED until the window is closed. Since we were on an SSH connection, there was no way for us to easily do this, meaning that the program was stuck in a halted state.
- We replaced plt.show() with plt.close(), and we assumed that this would fix the issue.

However, when we tried to run the CPU version, it was *painfully* slow.

⇒ That is, 6 seeds took 1hr of computation - since we iterate over this $25 * 4$ times, it follows that our final computation time would be over 4 days - that is not acceptable!!

This was much slower than our previous run - we think that it was due to increasing the patience (previous runs had terminated after only ~ 20 epochs, and we wanted better representation of long training)

⇒ We thought that using the GPU version would allow us to get more reasonable computation times. To that end, we returned to the GPU computation.

We already had everything in place for running this, since I had made the environment last night, and the issues last night had been very small problems... Ben fixed these:

- Error due to not deleting an old folder. Easily fixed.
- Error due to incorrect filename, easily fixed.
- Error due to number of workers, set this to zero (we think the old CUDA version of the GPU, 12.4, does not support this)

We also thought that running multiple GPU processes at once was causing them to slow down, so we put the resnet iteration on, and decided that we would just let this run.

⇒ Initially, it seemed to be quite slow (around 5 minutes), but after some time, this reduced to a good level (2-3 minutes).

8.3 Day 16 : Thursday, 19 November

???

CONFIRM NOTES WITH BEN!!

9 Week 9

9.1 Day 17 : Tuesday, 25 November

Our runs finally worked; the way we fixed the issue of maxing out the cpu was by including the line `torch.set_num_threads(x)` to the top, where x was the number of cores we wanted to use. For some reason, this would override slurm.

- Best MAE of ~ 109.22 , though this was surprising since we thought we had gotten better models previously. The hyperparameters were : 0 resnet layers, MAE loss, frequency sorting, and 0 hidden, length 50/50.
- The best first frequency MAE was ~ 97 - once again, this was surprising, since we were sure we had previously seen values below 90.

We wanted a way to quantify exactly how bad the intensity prediction was (we knew it was bad from observing output-target plots, but we needed a concrete way to prove this)

- We settled upon calculating the absolute MAE, and the variance of output and target intensities.
- I wrote a program to do this: 'analyser.py'.
- We found that the MAEs were very high (for frequency sorting, around 0.5), and that the output variances were very small (order 10^{-4}), while the target variances were much higher (order 0.1).
 - $\Rightarrow$ This confirmed what we had observed in output target plots : the model is terrible at intensity prediction, and just outputs constant values.

Having been surprised that our output MAEs were so high, we suspected that something might be wrong with the run.

- $\Rightarrow$ I looked at the definition of the `cr_cgcn` class a bit closer, and realised that the minimum number of hidden layers present at *any* given specification was 4, these were:
 - Conversion from the concatenated convolution to the hidden length (CGCNN)
 - Conversion from the hidden length to the resnet length (CGCNN)
 - Conversion from the resnet length to 32 (ResNet)
 - Conversion from 32 to the output length (ResNet)

This was unfortunate, since it meant that we had been underestimating the complexity of our previous results : if the trend was to be believed, then we could achieve better results by removing these extraneous layers.

I began by rewriting `cat_CGCNN` to output directly from the concatenated vector (i.e. 0 hidden layers, 0 resnet layers, one output layer after the concatenation).

- I put this in a new class, `direct_cat_CGCNN()`
 - $\Rightarrow$ Running this with the same HPs as our best run (LR=0.1, batch size=128, seed=152, $k = 0$, patience=5, momentum=0.9, `h_fea_len=50`, `fname='id_prop_fixed.npy'`, `lossmode='wloss'`)
 - $\Rightarrow$ We had a best average MAE of 108.3, with individual MAEs 93.2,103.6,102.99,133.6
 - $\Rightarrow$ The best first frequency MAE was 91.7 (which is a nice improvement)
- I also ran this using the MAE loss, which gave:
 - $\Rightarrow$ Average MAE of 110, with individual frequencies 93.9,101.9,106.0,138.3 after 26 epochs.

I decided (more or less on a whim) to test a new loss function, defined as:

$$\text{MAE}(\text{freq.}) + k * \text{MAE}(\text{freq} * \text{int})$$

I just had a feeling this would be good, and in fact it turned out to do quite well. We got our best result:

⇒ 2 hidden layers of 150, 0 resnet layers, $k=0.5$, all else the same.

⇒ AVERAGE MAE: 104.8

⇒ INDIV MAEs: 94.80687713623047, 107.37142944335938, 101.4595947265625, 115.6587142944336

⇒ BEST 1st FREQ MAE: 91.7

I tried this same model with the MAE loss:

Best single model:

At Epoch 23:

97.41240692138672

103.49041748046875

104.13424682617188

124.1497802734375

Best average MAE:

107.29671287536621

Best first freq:

95.30654907226562

Finally, I tried the same model with the weighted loss ($k = 0$):

Best single model:

At Epoch 37:

92.59329986572266

103.7417984008789

105.85018157958984

132.67625427246094

Best average MAE:

108.71538352966309

Best first freq:

92.59329986572266

This illustrates my inspiration for introducing the new loss function: the MAE loss tends to do better on each frequency, while the weighted loss excels on the first two frequencies.

⇒ The new loss combines both to achieve an overall better result.

Ben suggested that the second half of this loss function should actually be identical to the weighted loss with $k = 0$ - I thought this was a good idea, and decided to do the same run with this new loss function.

⇒ With the same seed, this returned:

Best single model:

At Epoch 23:

98.02642822265625

104.90753173828125

102.72089385986328

122.76611328125

Best average MAE:

107.1052417755127

Best 1st freq.

96.33587646484375

I was surprised that this performed worse than the initial attempt, so I decided to run again with seed 667.

```
Best single model:
At Epoch 34:
93.03072357177734
100.83399200439453
99.6796646118164
132.73666381835938
Best average MAE:
106.57026100158691
Best 1st freq.
93.03072357177734
```

This was a marginal improvement, but still not as good as I had hoped.

9.2 Wednesday, 26 November

We are reaching the end of the project, so I decided to explore one or two things in my own time.

A common theme throughout the previous few weeks was that the intensity prediction was completely nonsensical. This is illustrated in our plots, where the output-target visualisations reveal that the model is effectively outputting constant values for the intensities, i.e. making no useful predictions whatsoever.

⇒ We obviously want to do better than this, and it **should** be the case that our model is capable of predicting the eigenvalue intensities.

If we *could* predict the intensities, then one can imagine some sort of transfer learning model, where we predict either the intensities or frequencies first, then pass this information to a second model, which predicts the other parameter given this information.

⇒ As such, I think it would be a useful exercise to test the following:

- Can we actually predict the intensities directly from CGCNN?
- Can we predict the intensities if we pass the target frequencies as an input?

The first question is obviously a bit easier to consider, and if we are able to get some notional results, then it will imply that the second has potential, and is worth considering in more detail.

I added additional functionality to eig_hyps.py, now setting 'ints=True' in trainmodel() will pass only the intensity values to the model (the same id_prop as before can be used).

⇒ It makes sense to use intensity sorting here, since there is now no frequency information for the values to be compared to.

⇒ Normalisation is set to 1, and we use the simpleNormalizer() class (this has the distinguishing feature of normalising the entire target vector).

An initial run, with $n_h = 0$, h_fea_len=50, lr=0.1, L1Loss, and seed 667 gives poor results.

⇒ Once more, the intensities are predicted in a small range of bands, and there is clearly no attempt to recreate the actual trend, simply fitting to the average value.

plot?

⇒ NB there are some 'mistake' values where the first frequency has 0 intensity. Could be due to negative frequency removal?

⇒ I realised that this was indeed due to negative frequency removal. I had implemented renormalisation, but forgot to re-sort values by intensity after doing so. I fixed this in 'idpropmyself.py', and replaced id_prop_fixed_isort.npy with the corrected version.

The final loss on the first run was ~ 0.16 , and this fix improved it to 0.15.

- ⇒ However, the results were NOT better. In fact, it was actually worse, since now the prediction 'bands' had become thin lines.
- ⇒ Clearly the model is tending towards predicting the average value of the inputs : is there some way to avoid this?

plot?

Using the MSELoss instead of L1Loss had no improvement. The lines actually become thinner.

plot?

I tried many things. Nothing worked.

- ⇒ Used this for advice:

[https://www.researchgate.net/post/](https://www.researchgate.net/post/Neural-network-outputting-average-values-of-all-outputs-Am-I-doing-anything-wrong)

Neural-network-outputting-average-values-of-all-outputs-Am-I-doing-anything-wrong

An even worse sign was that the model exhibited precisely the same behaviour when the number of convolutional layers was reduced to 1, with no additional layers: it is clear that absolutely none of the crystal structure is being encoded in this output.

I'm not sure how we could possibly change this outcome.

- ⇒ For now, I'm going to give up on intensity prediction.

My final attempt was to modify newloss to train the outputs relative to the difference between output and target intensities.

- ⇒ It works well if trained relative to the difference in frequencies, so if we want to establish the correct relationship in intensities, it makes sense that we should attempt to base our frequency relation (which we know we can reconstruct to some extent) on the intensities.
- ⇒ Then, if we at all able to recover the correct frequency relation, we may be able to also recover the correct intensity relation: at least, this is the idea.
- ⇒ I did many runs with this loss function ('badloss'), and none of them yielded better results. Once more, the intensity outputs were constant (this surprised me, since I thought they would at least be distributed with some degree of randomness).
 - Again, the model just fit to the average values. And since the intensity relation was so terrible, the output frequencies were also just constant nonsense.
 - It is interesting how this happens when training via the intensity, but not via the frequency.
- ⇒ I did one final run, where I set the scale to 100 (I wanted the second term to be more important than the first, so that we hopefully recover the correct output).
 - ⇒ Even worse than before. Since minimising the second function takes precedent, it loses all sense of scale, and we end up with negative frequency and intensity values (mathematically this makes sense, since they will multiply to a positive number).
 - ⇒ It somehow manages to get even worse if I add a softplus at the end!

Overall : VERY UNSUCCESSFUL DAY!

9.3 Day 18 : Thursday, 27 November

We noticed various flaws with our fewlayers iteration (where we had reduced extraneous layers in the model)

- The form of newloss was wrong (we were using the worse version, where the second term is multiplied only by target intensity)
 - ⇒ Switched this to the correct version.
- general_cgcnn was using the wrong CGCNN core (cat_CGCNN instead of gen_CGCNN), which meant that we were adding two extra layers in the first half for no reason.
 - ⇒ This was easily fixed by changing cat_CGCNN to gen_CGCNN and running again.

We had noticed some inconsistencies between the results on our respective computers and the server...

- I had been using a scaling of 500 for my runs, whereas Ben had been using 4410 for his runs.
 - ⇒ We noticed that for newloss this made a BIG difference, and often would decide whether the model actually converged or not.
- Because of this, we decided to do our next iteration with scale 500 (as it seemed to be objectively better)
 - ⇒ It seemed like it made very little difference for both wloss2 and freqloss, whereas for newloss it was incredibly important.
 - ⇒ As such, our previous results should not be invalidated, but we should ensure that any newloss results are done at a lower scale.

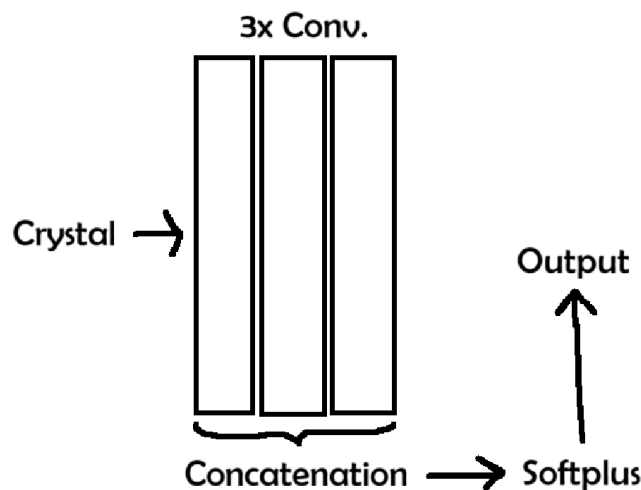
We decided to redo the iterations with scale 500, $n_h = [0,1,2]$ and $h_fea_len = [50, 90, 150, 250, 350]$.

⇒ We put this on for all of the loss functions, saving our results to runs_scale500.

We had a short meeting with our supervisor, who said that our results were satisfactory but that we should consider some additional analysis of the NN results.

⇒ Specifically, she mentioned using a 't-SNE' algorithm to cluster our CGCNN embeddings.

⇒ I wrote a modified class, trunc_cgcnn(), to output the CGCNN component, whose structure is:



⇒ Concatenation length is $n_conv * atom_fea_len$; for us this is $3 * 64 = 192$.

We would need to import the weights from a trained model into this, for which I would have to:

- (a) extract the state dictionary

- (b) chop off unimportant parts (everything except CGCNN part)
- (c) rename the keys to match `trunc_cgcnn()`
- (d) make a dummy instance of `trunc_cgcnn()`, using the `dummymodel()` command in `eig_hyps.py`
- (e) import the new state dict to truncated model
- (f) calculate the embeddings for the entire dataset using `find_embeddings()` in `eig_hyps.py`

This gives us a list of embeddings for a given trained model. The implementation is:

```
sd = model.state_dict()
keys = list(sd.keys())
# delete bad keys
for i,k in enumerate(keys[38:]):
    del sd[k]
# find new keys
trunc_model = eh.dummymodel(fname=fname)
newkeys = list(trunc_model.state_dict().keys())
# rename keys
td = OrderedDict((newkeys[i],v) for i,(k,v)
                  in enumerate(sd.items()))

# load state dict
trunc_model.load_state_dict(td)
embs = eh.find_embeddings(trunc_model,N=5000,fname=fname)
```

To do the t-SNE, we used the sklearn package (which i installed via `conda install conda_forge::sklearn`)

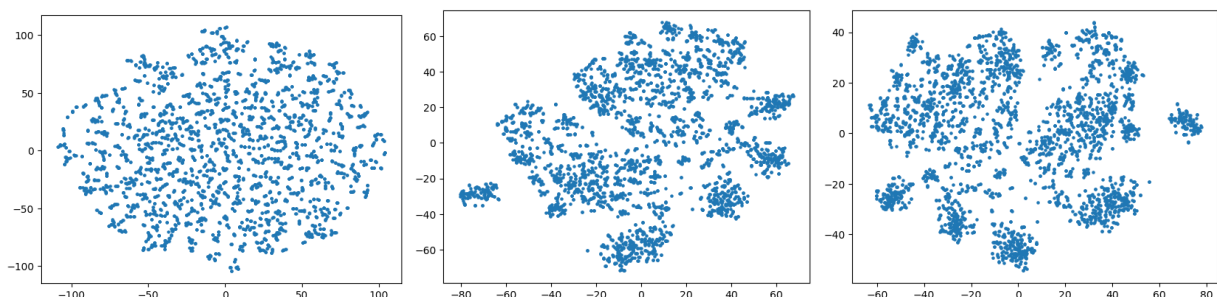
⇒ The exact command is `sklearn.manifold.TSNE()`

We can calculate the TSNE embedding in 2D via:

```
tsne_embs = TSNE(n_components=2, learning_rate='auto',
                 init='random', perplexity=50).fit_transform(embs)
```

We then plot this in a scatter plot.

⇒ We found that a high perplexity tended to work better for clustering. We tried 5, 30, 50 and found that 50 worked out.



Left to right: 5, 30, 50 perplexity.

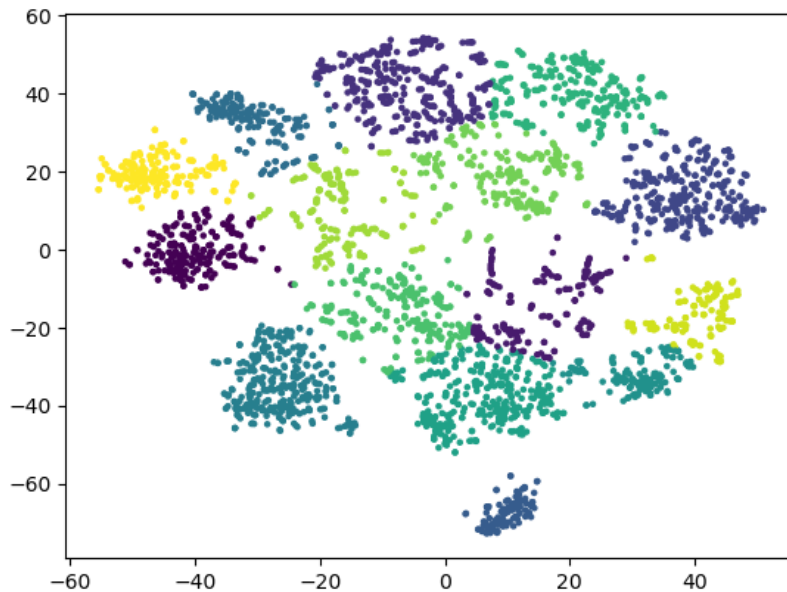
To identify the clusters, we decided to use `k_means`, which is imported from the same sklearn package as:

```
from sklearn.cluster import k_means
```

We used this to identify clusters as:

```
centroids, labels, _ = k_means(tsne_embs, n_clusters=15)
plt.scatter(tsne_embs[:,0], tsne_embs[:,1], c=labels, s=5)
```

The colours in the plot are dictated by the assigned cluster (from `k_means`).



Identified t-SNE clusters for a different model.

I found a very cool article talking about t-SNE interpretation:

<https://distill.pub/2016/misread-tsne/>

⇒ It seems like we will just have to compare certain properties of the crystals in each cluster, and attempt to match these to our embedded clusters... Maybe there is an easier way to do this?

I will base my t-SNE analysis off of the lines of reasoning followed in this paper, <https://arxiv.org/pdf/2504.20112>, when it comes time for it. That is, there are a few steps to talk about (once we have our best model):

1. Identifying relevant clusters.
2. Sampling crystals from each cluster.
3. Drawing connections between sampled crystals.
4. Retracing physical inferences from the groupings.

9.4 Friday, 28 November

The results of our iteration were **very** promising. I will summarise the key points here:

- Newloss gave the best results, and ALL of the complex models converged.
- ⇒ Interestingly, the models with 0,1 hidden layers performed worse than those with 2 hidden layers. This is the opposite of what we have seen previously, and is very promising, since it indicates that increasing the complexity may lead to better results.
- Our best model was newloss, 2 hidden layers, hidden feature length 50, and seed 393.
- ⇒ This had an MAE of 101.8 - much better than our previous iteration outcome!
- Every single model we tried with the other two loss functions properly converged (very nice).

9.5 Saturday, 28 November

After the success of our initial run, we had noticed that more complex models seemed to converge well with newloss - as such, we decided to do an iteration over larger values for n_h :

```

n_h = [3,4,5]
h_fea_len = [50,75,90,120,150,200]
scale = 500
# 6 seeds for each run

```

For completeness, we also did the same iteration with a single resnet layer, $n_r = 1$.

⇒ My testing had indicated that this would perform worse, but we would like quantifiable data to prove that this was in fact the case.

The python file was `bigrun3_wloss3.py`, and we saved to the following folders:

- `runs_scale500/wloss3_500_nr0`
- `runs_scale500/wloss3_500_nr1`

We used the `id_prop` on the server (which sorts from low to high in frequency). After the run finished, we made the following observations of its results:

- The best models had 3,5 hidden layers, and hidden feature length 50, with MAE 102, and seeds 393,471.
- None of the resnet models converged.
- The best MAEs were quite close, and all but the most complex model converged to MAEs near 105ish.
 - ⇒ We can probably put this fluctuation down to the stochastic nature of the optimisation.
 - ⇒ There were many very good models with MAE 101-105.
- Most models finished around 60-80 epochs, with some maxing out (indicating further training might lead to improvements).
 - Only the more complex models maxed outs.

9.6 Monday, 1 December

Wrote a script to optimise scale and try different seeds.

⇒ We decided to do this after noticing that reducing the scale to 500 had a big improvement : since this value was just an initial guess, it is possible that there are more optimum values out there.

⇒ The models we chose to consider were the best we had seen so far:

- 3,5 hidden layers, hidden feature length 50, 6 seeds for each.
- The scale values we iterated over were:

`[200,250,300,350,400,450,550,600,650,700,800,1000,2000,4410]`

- Filename : `bigrun4_wloss3_scale_seed.py`

The structure of the program was as follows:

1. Firstly, iterate over our different scale values for 3,5 layers.
2. Secondly, choose the best model and iterate this over 30 different seeds.

⇒ Using more seeds will likely improve the result, and give us an idea of the average performance of these models etc.

The seed iteration failed overnight (a typo or something), we decided to return to it tomorrow. However, the scale portion of the run seemed to succeed.

10 Week 10 (missing anything after Day 1)

10.1 Day 19 : Tuesday, 2 December

We decided to put some runs on the server to go while we were working on other things:

- An iteration of our best model over different scale values, so we could get a general idea of what was best.
- An iteration of the then best model over 30 seeds, which should give us an indicative idea of the stdev. of MAEs, as well as a nice survey of the possible seeds available.

The model we decided to put on was 3 hidden layers, of feature length 50, with newloss, $k = 0.5$.

- ⇒ This had gotten an MAE of 101.7 in our iteration over newloss models, and was one of the best.
- ⇒ In general, we found that sorting by frequency gives significantly better results than by intensity (around 120 vs 100).

The results of the scale run indicated that 500 was the best choice, so our previous results would be comparable.

- ⇒ However (for both the scale and in general) it seemed like other values were randomly distributed : it may just be that a certain seed has a favourable validation set for a particular scale, etc..
 - ⇒ that being said, we noticed that higher scales generally performed worse.
- ⇒ In hindsight, we should have used an object comparison set, e.g. a test set or something, to get exactly comparable results.
 - ⇒ But since we save the models we decided we would try to recalculate the MAEs using the whole dataset, so that we could get a more objective evaluation of each.

NB: choosing scale 500 from here on

We hadn't yet done an iteration over k values (0.5 had been my initial guess), so that we could get an idea for what worked best. Note that as $k \rightarrow 0$, the loss becomes identical to the MAE, so we chose the following value to iterate over:

[0.01,0.1,0.3,0.7,1,2]

note: not 0.5, we already tested this.

Furthermore, we decided that we would increase the patience of these runs, to ensure that they fully converged - we set patience to 15, and max epochs to 100 (for both the seed and k iterations).

- k iter files : bigrun5_wloss3_k.py (k iteration), SLURM_bigrun5_k.sh (SLURM), saved in wloss3_optim/wloss3_k
- seed iter files : bigrun4_scale_seed.py, SLURM_bigrun4_wloss3.sh, saved in wloss3_optim/wloss3_seeds

I decided to take our current best model and run it on mine (I will try retraining with the function I wrote to see if this can help us break the 100 MAE threshold).

- ⇒ I encountered some issues with my current code : it seemed as if the model being returned by trainmodel() was not actually the "best model"..
- ⇒ That is, I trained a model which gave the following results (as part of its MAE vector):

Best MAEs:
93.55973052978516
102.15911865234375

```

95.93267822265625
117.0285873413086
Best single model:
At Epoch 52:
94.8264389038086
103.76717376708984
95.93267822265625
118.00446319580078
Best average MAE:
103.13268852233887

```

⇒ However, when I then calculated the MAE using the same validation set, I got the following:

```

Individual means:
Freq. 1 : 96.0798054981527
Freq. 2 : 104.79118200241466
Freq. 3 : 98.48021890092066
Freq. 4 : 120.89018319853655
Overall mean : 105.06034740000614

```

This MAE was similar to the final MAE shown in the model training (105.38). While not exactly identical, this indicates that the model being returned may not actually be our 'best model', which is very concerning.

⇒ I thought that issue might be that I was saving a pointer to the state dictionary, rather than a distinct copy so I made the following change in eig_hyps.py:

Before:

```

if avg_mae < best_mae:
    best_mae = avg_mae
    best_sd = model.state_dict()

```

After:

```

if avg_mae < best_mae:
    best_mae = avg_mae
    best_sd = model.state_dict().copy()

```

However, I faced the same issue (apparently). Could it be that my MAE calculation in testmodel() differs from that in the training?? I checked and this seems not to be the case : though I use a batch size of 1 in testmodel as opposed to 128 in trainmodel, this shouldn't affect the output as I've been careful to remove 'nan' values in the final collection:

```

# find the maes this way:
f1s = testmaes[:,0].flatten()
f2s = testmaes[:,1].flatten()
f3s = testmaes[:,2].flatten()
f4s = testmaes[:,3].flatten()
# remove nans and take means
f1mean = np.mean(f1s[~np.isnan(f1s)])
f2mean = np.mean(f2s[~np.isnan(f2s)])
f3mean = np.mean(f3s[~np.isnan(f3s)])
f4mean = np.mean(f4s[~np.isnan(f4s)])
overall = np.mean([f1mean, f2mean, f3mean, f4mean])
# print
print("Individual means:")

```

```

print("Freq. 1: ", f1mean)
print("Freq. 2: ", f2mean)
print("Freq. 3: ", f3mean)
print("Freq. 4: ", f4mean)
print("Overall mean: ", overall)

```

To debug whether this is an issue with the state dictionary, I decided to do the following tests:

- Return the state dict copy as well as the final model, and see if they are the same.
- If they are, then the `.copy()` method is not working.

In fact, they **were** the same, as evidenced by the following tests, where I started with the final model:

```

Individual means:
Freq. 1 : 101.03246462378453
Freq. 2 : 110.94711211258075
Freq. 3 : 104.01642981492475
Freq. 4 : 111.52323234026977
Overall mean : 106.87980972288996

```

```
>> model.load_state_dict(bestsd)
```

```

Individual means:
Freq. 1 : 101.03246462378453
Freq. 2 : 110.94711211258075
Freq. 3 : 104.01642981492475
Freq. 4 : 111.52323234026977
Overall mean : 106.87980972288996

```

So clearly the `state_dict` assignment is NOT working: either the conditional is broken, or `model.state_dict().copy()` still returns a dictionary of pointers to each tensor (the state dict is a dictionary of tensors).

- ⇒ To attempt to solve this problem, I just need to clone and detach each sub-tensor of the dictionary. This can be achieved by defining a new `OrderedDict` (this is the format of the state dict), where we apply this operation to each item as follows:

```

sdtest = OrderedDict((k, v.clone().detach()) for i,
                      (k, v) in enumerate(sd.items())

```

- ⇒ Here, `sd` is the dictionary we want to copy, and `sdtest` should be fully detached.

- ⇒ Testing appears to show that this works?

I ran the following model to test this:

```

seed = 393
scale = 550
fname = 'id_prop_fixed.npy'
mae, loss, model = eh.trainModel(seed=seed, lr=0.1, infin=True,
                                   scale=scale, b
                                   n_h=3, h_fea_len=50, testing=False, fname=fname
                                   res=True, lossmode='n
                                   n_r=0
                                   k=0.5

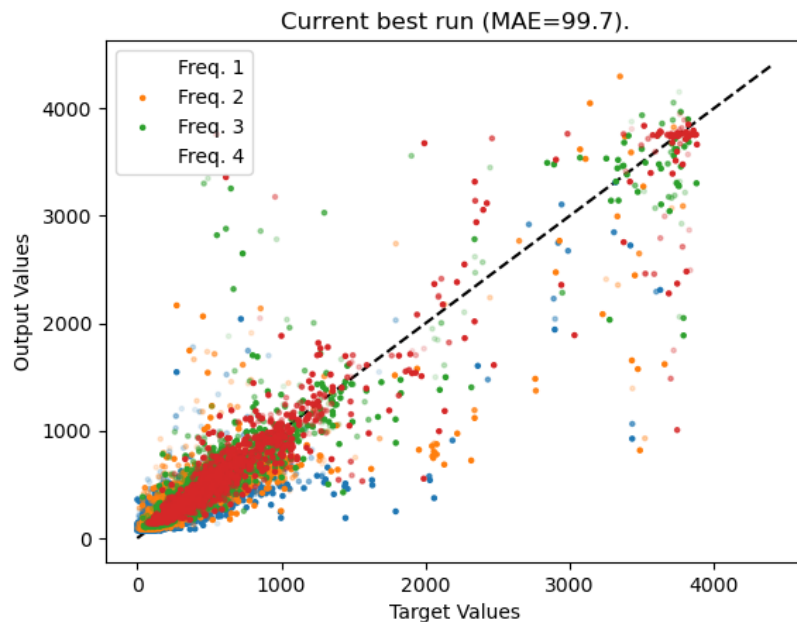
```

The results seemed to be very good for this run, actually:

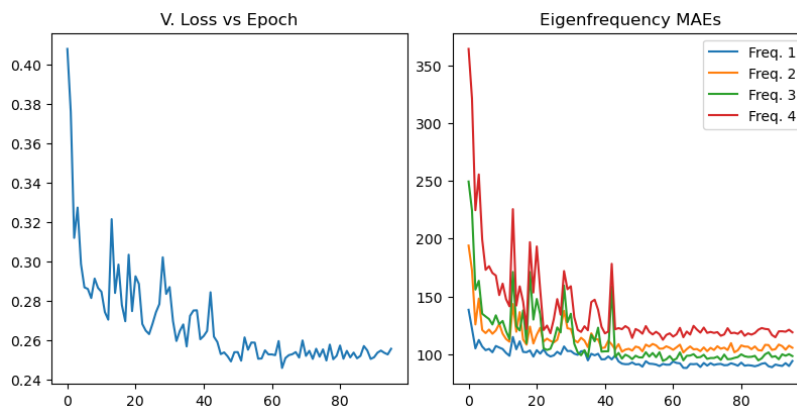
Best MAEs:
88.01910400390625
101.45177459716797
94.15789794921875
112.4323959350586
Best single model:
At Epoch 63:
88.16358184814453
101.45177459716797
94.62810516357422
114.6148681640625
Best average MAE:
99.7145824432373

Observations:

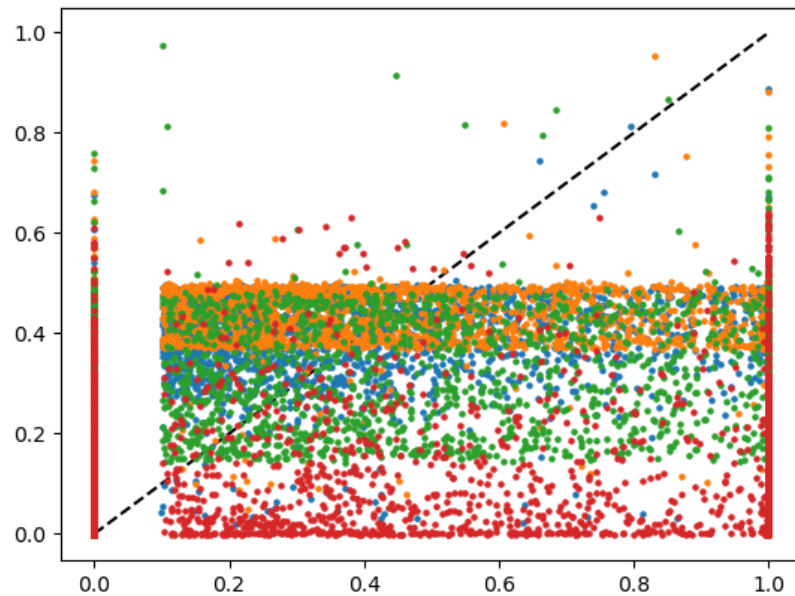
⇒ Our lowest MAE (according to trainmodel) was below 100 !! This is a huge step.



⇒ Interestingly, the third MAE is lower than the second : this is a general feature of our newloss models. Best model at 63 epochs.

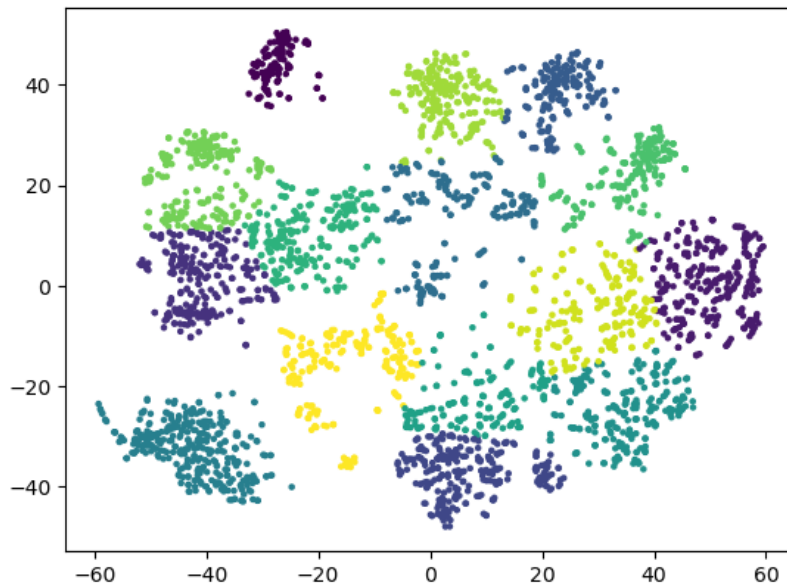


⇒ The intensities are still nonsense, this seems to be an issue we can not fix.



⇒ the t-SNE plot seems to have well-separated clusters, which we will analyse later.

⇒ K-Means does well to recognise clusters.



However, once more, when I calculate with testmodel, the MAE is different:

```
* full dataset *
Individual means:
Freq. 1 : 96.13423275125437
Freq. 2 : 104.34751123321251
Freq. 3 : 100.05878447134198
Freq. 4 : 107.85451029055912
Overall mean : 102.098759686592

* validation set *
Individual means:
Freq. 1 : 90.72832362644444
Freq. 2 : 105.12274342459419
Freq. 3 : 97.09133353878937
Freq. 4 : 118.17864827906831
Overall mean : 102.78026221722408
```

Why is this happening?? Time to find out!

Before getting into more debugging, I decided to test whether the model returning actually was a real issue, or if it was just an artifice of the MAE calculation - I returned a model using the old method (`best_sd = model.state_dict()`), and using the new method, and they were in fact different:

```
* old method *
Individual means:
Freq. 1 : 97.986609184336
Freq. 2 : 108.00397402490407
Freq. 3 : 105.28613695339466
Freq. 4 : 125.09744517008464
Overall mean : 109.09354133317984

* new method *
Individual means:
Freq. 1 : 97.94489280062932
Freq. 2 : 108.2468694545355
Freq. 3 : 103.31513952585976
Freq. 4 : 122.18454758664394
Overall mean : 107.92286234191712
```

So, this issue will have affected our previous results. The model saved (and used to calculate plots etc) will *not* be the best model. However, the MAEs should still be valid.

I later discovered the issue...

⇒ It was actually to do with the MAE calculation during training, where I had used the following code:

NB all variables initialised to zero each epoch.

```
# iterating over epochs:
vmae1 = mae(denormed_outs , denormed_ins , 0)
if vmae1 == 0:
    nc1 += 1
running_vmae1 += vmae1
vmae2 = mae(denormed_outs , denormed_ins , 1)
if vmae2 == 0:
    nc2 += 1
running_vmae2 += vmae2
vmae3 = mae(denormed_outs , denormed_ins , 2)
if vmae3 == 0:
    nc3 += 1
running_vmae3 += vmae3
vmae4 = mae(denormed_outs , denormed_ins , 3)
if vmae4 == 0:
    nc4 += 1
running_vmae4 += vmae4
avg_vmae1 = running_vmae1 / (i - nc1 + 1)
avg_vmae2 = running_vmae2 / (i - nc2 + 1)
avg_vmae3 = running_vmae3 / (i - nc3 + 1)
avg_vmae4 = running_vmae4 / (i - nc4 + 1)
```

However, this had a problem : for large batch sizes, the final batch would have fewer crystals than the other batches, and where there were no eigenvalues, the number would also be reduced (with this effect being especially prominent for the fourth eigenvalue)

⇒ This meant that the crystals were not being weighted properly, and that the final value was prone to random fluctuations depending on the model, seed, and batch size.

Why I programmed it like this the first time is a mystery to me, but I fixed it with the following changes:

- Augmenting the mae() function to output the both the MAE and the number of crystals considered : mae() -> (MAE, n)
- Weighting the contribution per epoch by n
- Summing n for each epoch in the nc# variables
- Dividing the final sum of n*MAE by nc#.

Clearly, this should equally weight all considered crystals, and give a consistent output independent of batch size : more importantly, an objective comparison between models for the same seed !

⇒ The new code is:

```
# iteration over epochs
vmae1,n1 = mae(denormed_outs , denormed_ins , 0)
nc1 += n1
running_vmae1 += n1*vmae1
vmae2,n2 = mae(denormed_outs , denormed_ins , 1)
nc2 += n2
running_vmae2 += n2*vmae2
vmae3,n3 = mae(denormed_outs , denormed_ins , 2)
nc3 += n3
running_vmae3 += n3*vmae3
vmae4,n4 = mae(denormed_outs , denormed_ins , 3)
nc4 += n4
running_vmae4 += n4*vmae4
# after iteration
avg_vmae1 = running_vmae1 / (nc1)
avg_vmae2 = running_vmae2 / (nc2)
avg_vmae3 = running_vmae3 / (nc3)
avg_vmae4 = running_vmae4 / (nc4)
```

We will use this going forwards, and recalculate any MAEs we use in the report.

⇒ Thankfully, the error is only ~ 1% on average, so our inferences should still hold correct.